

axiomTM



The 30 Year Horizon

<i>Manuel Bronstein</i>	<i>William Burge</i>	<i>Timothy Daly</i>
<i>James Davenport</i>	<i>Michael Dewar</i>	<i>Martin Dunstan</i>
<i>Albrecht Fortenbacher</i>	<i>Patrizia Gianni</i>	<i>Johannes Grabmeier</i>
<i>Jocelyn Guidry</i>	<i>Richard Jenks</i>	<i>Larry Lambe</i>
<i>Michael Monagan</i>	<i>Scott Morrison</i>	<i>William Sit</i>
<i>Jonathan Steinbach</i>	<i>Robert Sutor</i>	<i>Barry Trager</i>
<i>Stephen Watt</i>	<i>Jim Wen</i>	<i>Clifton Williamson</i>

Volume 13: Proving Axiom Correct

Portions Copyright (c) 2005 Timothy Daly

The Blue Bayou image Copyright (c) 2004 Jocelyn Guidry

Portions Copyright (c) 2004 Martin Dunstan

Portions Copyright (c) 2007 Alfredo Portes

Portions Copyright (c) 2007 Arthur Ralfs

Portions Copyright (c) 2005 Timothy Daly

Portions Copyright (c) 1991-2002,
The Numerical ALgorithms Group Ltd.
All rights reserved.

This book and the Axiom software is licensed as follows:

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of The Numerical ALgorithms Group Ltd. nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Inclusion of names in the list of credits is based on historical information and is as accurate as possible. Inclusion of names does not in any way imply an endorsement but represents historical influence on Axiom development.

Michael Albaugh	Cyril Alberga	Roy Adler
Christian Aistleitner	Richard Anderson	George Andrews
S.J. Atkins	Jeremy Avigad	Henry Baker
Martin Baker	Stephen Balzac	Yurij Baransky
David R. Barton	Thomas Baruchel	Gerald Baumgartner
Gilbert Baumslag	Michael Becker	Nelson H. F. Beebe
Jay Belanger	David Bindel	Fred Blair
Vladimir Bondarenko	Mark Botch	Raoul Bourquin
Alexandre Bouyer	Karen Braman	Wolfgang Brehm
Peter A. Broadbery	Martin Brock	Manuel Bronstein
Stephen Buchwald	Florian Bundschuh	Luanne Burns
William Burge	Ralph Byers	Quentin Carpent
Robert Cavines	Bruce Char	Ondrej Certik
Tzu-Yi Chen	Bobby Cheng	Cheekai Chin
David V. Chudnovsky	Gregory V. Chudnovsky	Mark Clements
James Cloos	Jia Zhao Cong	Josh Cohen
Christophe Conil	Don Coppersmith	George Corliss
Robert Corless	Gary Cornell	Meino Cramer
Jeremy Du Croz	David Cyganski	Nathaniel Daly
Timothy Daly Sr.	Timothy Daly Jr.	James H. Davenport
David Day	James Demmel	Didier Deshommes
Michael Dewar	Inderjit Dhillon	Jack Dongarra
Jean Della Dora	Gabriel Dos Reis	Claire DiCrescendo
Sam Dooley	Zlatko Drmac	Lionel Ducos
Iain Duff	Lee Duhem	Martin Dunstan
Brian Dupee	Dominique Duval	Robert Edwards
Heow Eide-Goodman	Lars Erickson	Mark Fahey
Richard Fateman	Bertfried Fauser	Stuart Feldman
John Fletcher	Brian Ford	Albrecht Fortenbacher
George Frances	Constantine Frangos	Timothy Freeman
Korrinn Fu	Marc Gaetano	Rudiger Gebauer
Van de Geijn	Kathy Gerber	Patricia Gianni
Gustavo Goertkin	Samantha Goldrich	Holger Gollan
Teresa Gomez-Diaz	Laureano Gonzalez-Vega	Stephen Gortler
Johannes Grabmeier	Matt Grayson	Klaus Ebbe Grue
James Griesmer	Vladimir Grinberg	Oswald Gschnitzer
Ming Gu	Jocelyn Guidry	Gaetan Hache
Steve Hague	Satoshi Hamaguchi	Sven Hammarling
Mike Hansen	Richard Hanson	Richard Harke
Bill Hart	Vilya Harvey	Martin Hassner
Arthur S. Hathaway	Dan Hatton	Waldek Hebisch
Karl Hegbloom	Ralf Hemmecke	Henderson
Antoine Hersen	Nicholas J. Higham	Roger House
Gernot Hueber	Pietro Iglio	Alejandro Jakubi
Richard Jenks	Bo Kagstrom	William Kahan
Kyriakos Kalorkoti	Kai Kaminski	Grant Keady
Wilfrid Kendall	Tony Kennedy	David Kincaid

Ted Kosan	Paul Kosinski	Igor Kozachenko
Fred Krogh	Klaus Kusche	Bernhard Kutzler
Tim Lahey	Larry Lambe	Kaj Laurson
Charles Lawson	George L. Legendre	Franz Lehner
Frederic Lehobey	Michel Levaud	Howard Levy
J. Lewis	Ren-Cang Li	Rudiger Loos
Craig Lucas	Michael Lucks	Richard Luczak
Camm Maguire	Francois Maltey	Osni Marques
Alasdair McAndrew	Bob McElrath	Michael McGettrick
Edi Meier	Ian Meikle	David Mentre
Victor S. Miller	Gerard Milmeister	Mohammed Mobarak
H. Michael Moeller	Michael Monagan	Marc Moreno-Maza
Scott Morrison	Joel Moses	Mark Murray
William Naylor	Patrice Naudin	C. Andrew Neff
John Nelder	Godfrey Nolan	Arthur Norman
Jinzhong Niu	Michael O'Connor	Summat Oemrawsingh
Kostas Oikonomou	Humberto Ortiz-Zuazaga	Julian A. Padget
Bill Page	David Parnas	Susan Pelzel
Michel Petitot	Didier Pinchon	Ayal Pinkus
Frederick H. Pitts	Jose Alfredo Portes	E. Quintana-Orti
Gregorio Quintana-Orti	Beresford Parlett	A. Petitot
Peter Poromaa	Claude Quitte	Arthur C. Ralfs
Norman Ramsey	Anatoly Raportirenko	Guilherme Reis
Huan Ren	Albert D. Rich	Michael Richardson
Jason Riedy	Renaud Rioboo	Jean Rivlin
Nicolas Robidoux	Simon Robinson	Raymond Rogers
Michael Rothstein	Martin Rubey	Jeff Rutter
Philip Santas	Alfred Scheerhorn	William Schelter
Gerhard Schneider	Martin Schoenert	Marshall Schor
Frithjof Schulze	Fritz Schwarz	Steven Segletes
V. Sima	Nick Simicich	William Sit
Elena Smirnova	Jacob Nyffeler Smith	Ken Stanley
Jonathan Steinbach	Fabio Stumbo	Christine Sundaresan
Robert Sutor	Moss E. Sweedler	Eugene Surowitz
Max Tegmark	T. Doug Telford	James Thatcher
Laurent Thery	Balbir Thomas	Mike Thomas
Dylan Thurston	Francoise Tisseur	Steve Toleque
Raymond Toy	Barry Trager	Themos T. Tsikas
Gregory Vanuxem	Kresimir Veselic	Christof Voemel
Bernhard Wall	Stephen Watt	Jaap Weel
Juergen Weiss	M. Weller	Mark Wegman
James Wen	Thorsten Werther	Michael Wester
R. Clint Whaley	James T. Wheeler	John M. Wiley
Berhard Will	Clifton J. Williamson	Stephen Wilson
Shmuel Winograd	Robert Wisbauer	Sandra Wityak
Waldemar Wiwianka	Knut Wolf	Yanyang Xiao
Liu Xiaojun	Clifford Yapp	David Yun
Qian Yun	Vadim Zhytnikov	Richard Zippel
Evelyn Zoernack	Bruno Zuercher	Dan Zwillinger

Contents

1	Here is a problem	3
1.1	Setting up the problem	3
1.2	Axiom NNI GCD	4
1.3	Mathematics	6
1.4	Approaches	8
2	Theory	11
2.1	The Division Algorithm	12
3	Software Details	15
3.1	Installed Software	15
4	Temporal Logic of Actions (TLA)	17
4.1	The algorithm	17
	Creating a new TLA+ module	18
	Definitions	18
	Constants and variables	19
	The specification	19
	Summary	20
4.2	A simple proof	20
	The invariant	20
	Checking proofs	20
	Using facts and definitions	21
4.3	Divisibility Definition	22
5	COQ proof of GCD	25
5.1	Basics of the Calculus of Constructions	25
	Terms	25
	Judgements	26
	Inference Rules	26
	Defining Logical Operators	27
	Defining Types	27
5.2	Why does COQ have Prop?	27
5.3	Source code of COQ GCD Proof	28

6	LEAN proof of GCD	37
7	Formal Pre- and Post-conditions	45
8	Types and Signatures	47
9	COQ nat vs Axiom NNI	51
	Library Coq.Init.Nat	51
10	Other Ideas to Explore	59
A	The Global Environment	61
	Bibliography	63
	Index	73

New Foreword

On October 1, 2001 Axiom was withdrawn from the market and ended life as a commercial product. On September 3, 2002 Axiom was released under the Modified BSD license, including this document. On August 27, 2003 Axiom was released as free and open source software available for download from the Free Software Foundation's website, Savannah.

Work on Axiom has had the generous support of the Center for Algorithms and Interactive Scientific Computation (CAISS) at City College of New York. Special thanks go to Dr. Gilbert Baumslag for his support of the long term goal.

The online version of this documentation is roughly 1000 pages. In order to make printed versions we've broken it up into three volumes. The first volume is tutorial in nature. The second volume is for programmers. The third volume is reference material. We've also added a fourth volume for developers. All of these changes represent an experiment in print-on-demand delivery of documentation. Time will tell whether the experiment succeeded.

Axiom has been in existence for over thirty years. It is estimated to contain about three hundred man-years of research and has, as of September 3, 2003, 143 people listed in the credits. All of these people have contributed directly or indirectly to making Axiom available. Axiom is being passed to the next generation. I'm looking forward to future milestones.

With that in mind I've introduced the theme of the "30 year horizon". We must invent the tools that support the Computational Mathematician working 30 years from now. How will research be done when every bit of mathematical knowledge is online and instantly available? What happens when we scale Axiom by a factor of 100, giving us 1.1 million domains? How can we integrate theory with code? How will we integrate theorems and proofs of the mathematics with space-time complexity proofs and running code? What visualization tools are needed? How do we support the conceptual structures and semantics of mathematics in effective ways? How do we support results from the sciences? How do we teach the next generation to be effective Computational Mathematicians?

The "30 year horizon" is much nearer than it appears.

Tim Daly
CAISS, City College of New York
November 10, 2003 ((iHy))

Ultimately we would like Axiom to be able to prove that an algorithm generates correct results. There are many steps between here and that goal, including proving one Axiom algorithm correct through all of the levels from Spad code, to the Lisp code, to the C code, to the machine code; a daunting task of its own.

The proof of a single Axiom algorithm is done with an eye toward automating the process. Automated machine proofs are not possible in general but will exist for known algorithms.

Writing is nature's way of letting you know how sloppy your thinking is – Guindon[Lamp02]

Mathematics is nature's way of letting you know how sloppy your writing is. – Leslie Lamport[Lamp02]

The existence of the computer is giving impetus to the discovery of algorithms that generate proofs. I can still hear the echos of the collective sigh of relief that greeted the announcement in 1970 that there is no general algorithm to test for integer solutions to polynomial Diophantine equations; Hilbert's tenth problem has no solution. Yet, as I look at my own field, I see that creating algorithms that generate proofs constitutes some of the most important mathematics being done. The all-purpose proof machine may be dead, but tightly targeted machines are thriving. – Dave Bressoud [Bres93]

In contrast to humans, computers are good at performing formal processes. There are people working hard on the project of actually formalizing parts of mathematics by computer, with actual formally correct formal deductions. I think this is a very big but very worthwhile project, and I am confident that we will learn a lot from it. The process will help simplify and clarify mathematics. In not too many years, I expect that we will have interactive computer programs that can help people compile significant chunks of formally complete and correct mathematics (based on a few perhaps shaky but at least explicit assumptions) and that they will become part of the standard mathematicians's working environment. – William P. Thurston [Thur94]

Our basic premise is that the ability to construct and modify programs will not improve without a new and comprehensive look at the entire programming process. Past theoretical research, say, in the logic of programs, has tended to focus on methods for reasoning about individual programs; little has been done, it seems to us, to develop a sound understanding of the process of programming – the process by which programs evolve in concept and in practice. At present, we lack the means to describe the techniques of program construction and improvement in ways that properly link verification, documentation and adaptability.

– Scherlis and Scott (1983) in [Maso86]

...constructive mathematics provides a way of viewing the language of logical propositions as a *specification* language for programs. An ongoing thrust of work in computer science has been to develop program specification languages and formalisms for systematically deriving programs from specifications. For constructive mathematics to provide such a methodology, techniques are needed for systematically extracting programs from constructive proofs. Early work in this field includes that of Bishop and Constable. What distinguished Martin-Löf's '82 type theory was that the method it suggested for program synthesis was exceptionally simple: a direct correspondence was set up between the constructs of mathematical logic, and the constructs of a functional programming language. Specifically, every proposition was considered to be isomorphic to a type expression, and the proof of a proposition would suggest precisely how to construct an inhabitant of the type, which would be a term in a functional programming language. The term that inhabits the type corresponding to a proposition is often referred to as the *computational content* of the proposition.

– Paul Bernard Jackson[[Jack95](#)]

Chapter 1

Here is a problem

Proving programs correct involves working with a second programming language, the proof language, that is well-founded on some theory. Proofs (programs), can be reduced (compiled) in this new language to the primitive constructs (machine language).

The ideal case would be that the programming language used, such as Spad, can be isomorphic, or better yet, syntactically the same as the proof language. Unfortunately that is not (yet?) the case with Spad.

The COQ system language, Gallina, is the closest match to Spad.

1.1 Setting up the problem

The GCD function will be our first example of a proof.

The goal is to prove that Axiom's implementation of the Euclidean GCD algorithm is correct.

We need to be clear about what is to be proven. In this case, we need to show that, given $\text{GCD}(a,b)$,

1. **GCD** is a function from $a \times b \Rightarrow c$
2. **a** and **b** are elements of the correct type
3. **c**, the result, is the correct type
4. the meaning of **divisor**
5. the meaning of a **common divisor**
6. **GCD** terminates

We next need to set up the things we know in "the global environment", generally referred to as **E** in Coq.

Axiom's GCD is categorically defined to work over any Euclidean domain. This means that the axioms of a Euclidean domain are globally available. In fact, this is stronger than we need since

- commutative rings $\subset$ integral domains
- integral domains $\subset$ integrally closed domains
- integrally closed domains $\subset$ GCD domains
- GCD domains $\subset$ unique factorization domains
- unique factorization domains $\subset$ principal ideal domains
- principal ideal domains $\subset$ Euclidean domains

A Euclidean function on R is a function f from $\mathbb{R} \setminus \{0\}$ to the non-negative integers satisfying the following fundamental division-with-remainder property[[WikiED](#)]:

$D(a, b)$ = set of common divisors of a and b .

$\gcd(a, b) = \max D(a, b)$

1.2 Axiom NNI GCD

NonNegativeInteger inherits `gcd` from `Integer` up the “add chain” since it is a subtype of `Integer`. `Integer` has `EuclideanDomain` as an ancestor[[Book103](#)]:

```
(1) -> getAncestors "Integer"
```

```
(1)
{AbelianGroup, AbelianMonoid, AbelianSemiGroup, Algebra, BasicType,
 BiModule, CancellationAbelianMonoid, CharacteristicZero, CoercibleTo,
 CombinatorialFunctionCategory, CommutativeRing, ConvertibleTo,
 DifferentialRing, EntireRing, EuclideanDomain, GcdDomain,
 IntegerNumberSystem, IntegralDomain, LeftModule, LeftOreRing,
 LinearlyExplicitRingOver, Module, Monoid, OpenMath, OrderedAbelianGroup,
 OrderedAbelianMonoid, OrderedAbelianSemiGroup,
 OrderedCancellationAbelianMonoid, OrderedIntegralDomain, OrderedRing,
 OrderedSet, PatternMatchable, PrincipalIdealDomain, RealConstant,
 RetractableTo, RightModule, Ring, Rng, SemiGroup, SetCategory, StepThrough,
 UniqueFactorizationDomain}
```

Type: Set(Symbol)

From category `EuclideanDomain` (EUCDOM) we find the implementation of the Euclidean GCD algorithm[[Book102](#)]:

```

gcd(x,y) ==                                --Euclidean Algorithm
  x:=unitCanonical x
  y:=unitCanonical y
  while not zero? y repeat
    (x,y):= (y,x rem y)
    y:=unitCanonical y  -- this doesn't affect the
                        -- correctness of Euclid's algorithm,
                        -- but
                        -- a) may improve performance
                        -- b) ensures gcd(x,y)=gcd(y,x)
                        -- if canonicalUnitNormal

x

```

The `unitCanonical` function comes from the category `IntegralDomain` (INTDOM) where we find:

```

unitNormal: % -> Record(unit:%,canonical:%,associate:%)
  ++ unitNormal(x) tries to choose a canonical element
  ++ from the associate class of x.
  ++ The attribute canonicalUnitNormal, if asserted, means that
  ++ the "canonical" element is the same across all associates of x
  ++ if \spad{unitNormal(x) = [u,c,a]} then
  ++ \spad{u*c = x}, \spad{a*u = 1}.
unitCanonical: % -> %
  ++ \spad{unitCanonical(x)} returns \spad{unitNormal(x).canonical}.

```

implemented as

```

UCA ==> Record(unit:%,canonical:%,associate:%)
if not (% has Field) then
  unitNormal(x) == [1$,x,1$]$UCA -- the non-canonical definition
unitCanonical(x) == unitNormal(x).canonical -- always true
recip(x) == if zero? x then "failed" else _exquo(1$,x)
unit?(x) == (recip x case "failed" => false; true)
if % has canonicalUnitNormal then
  associates?(x,y) ==
    (unitNormal x).canonical = (unitNormal y).canonical
else
  associates?(x,y) ==
    zero? x => zero? y
    zero? y => false
    x exquo y case "failed" => false
    y exquo x case "failed" => false
    true

```

Coq proves the following GCD function:

```

Fixpoint gcd a b :=
  match a with

```

```

    | 0 => b
    | S a' => gcd (b mod (S a')) (S a')
end.

```

This can be translated directly to working Spad code:

```

GCD(x:NNI,y:NNI):NNI ==
  zero? x => y
  GCD(y rem x,x)

```

with the test case results of:

```

(1) -> GCD(2415,945)
      Compiling function mygcd2 with type (NonNegativeInteger,
      NonNegativeInteger) -> NonNegativeInteger

(1)  105
                                           Type: PositiveInteger
(2) -> GCD(0,945)

(2)  945
                                           Type: PositiveInteger
(3) -> GCD(2415,0)

(3)  2415
                                           Type: PositiveInteger
(4) -> GCD(17,15)

(4)  1
                                           Type: PositiveInteger

```

1.3 Mathematics

From Buchberger[Buch97],

Define “divides”

$$t|a \iff \exists u(t \cdot u = a)$$

Define “greatest common divisor”

$$\text{GCD}(a, b) = \forall t \max(t|a \wedge t|b)$$

Theorem:

$$(t|a \wedge t|b) \iff t|(a - b) \wedge t|b$$

Euclids’ Algorithm

$$a > b \Rightarrow \text{GCD}(a, b) = \text{GCD}(a - b, b)$$

By the definition of GCD we need to show that

$$\forall t \max(t|a \wedge t|b) = \forall t \max(t|(a-b) \wedge t|b)$$

Thus we need to show that

$$(t|a \wedge t|b) \iff (t|(a-b) \wedge t|b)$$

Let t be arbitrary but fixed and assume

$$(t|a \wedge t|b) \tag{1.1}$$

We have to show

$$t|(a-b) \tag{1.2}$$

and

$$t|b \tag{1.3}$$

Equation 1.3 follows propositionally. For equation 1.2, by definition of “divides”, we have to find a w such that

$$t \cdot w = a - b \tag{1.4}$$

From 1.1, by definition of “divides”, we know that for certain u and v

$$t \cdot u = a$$

and

$$t \cdot v = b$$

Hence,

$$a - b = t \cdot u - t \cdot v$$

But

$$t \cdot u - t \cdot v = t \cdot (u - v)$$

So we need to find

$$w = u - v$$

and

Find w such that $t \cdot u - t \cdot v = t \cdot w$

1.4 Approaches

There are several systems that could be applied to approach the proof.

The plan is to initially look at Coq and ACL2. Coq seems to be applicable at the Spad level. ACL2 seems to be applicable at the Lisp level. Both levels are necessary for a proper proof.

Coq is very close to Spad in spirit so we can use it for the high-level proofs.

ACL2 is a Lisp-level proof technology which can be used to prove the Spad-to-Lisp level.

There is an LLVM to ACL2 translator which can be used to move from the GCL Lisp level to the hardware since GCL compiles to C. In particular, the "Vellvm: Verifying the LLVM" [Zdan14] project is important.

Quoting from Hardin [Hard14]

LLVM is a register-based intermediate in Static Single Assignment (SSA) form. As such, LLVM supports any number of registers, each of which is only assigned once, statically (dynamically, of course, a given register can be assigned any number of times). Appel has observed that "SSA form is a kind of functional programming"; this observation, in turn, inspired us to build a translator from LLVM to the applicative subset of Common Lisp accepted by the ACL2 theorem prover. Our translator produces an executable ACL2 specification that is able to efficiently support validation via testing, as the generated ACL2 code features tail recursion, as well as in-place updates via ACL2's single-threaded object (stobj) mechanism. In order to ease the process of proving properties about these translated functions, we have also developed a technique for reasoning about tail-recursive ACL2 functions that execute in-place, utilizing a formally proven "bridge" to primitive-recursive versions of those functions operating on lists.

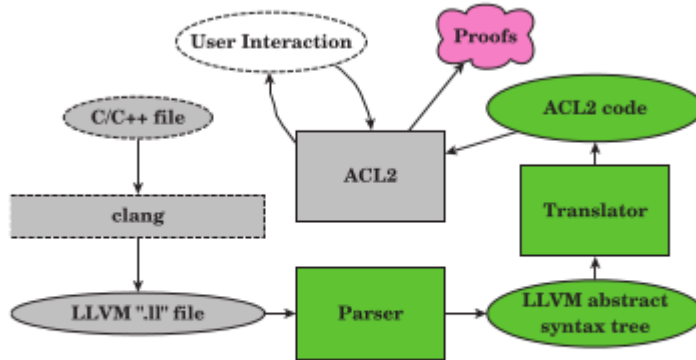


Figure 1: LLVM-to-ACL2 translation toolchain.

Hardin [Hard13] describes the toolchain thus:

Our translation toolchain architecture is shown in Figure 1. The left side of the figure depicts a typical compiler frontend producing LLVM intermediate code. LLVM output can be produced either as a binary “bitcode” (.bc) file, or as text (.ll file). We chose to parse the text form, producing an abstract syntax tree (AST) representation of the LLVM program. Our translator then converts the AST to ACL2 source. The ACL2 source file can then be admitted into an ACL2 session, along with conjectures that one wishes to prove about the code, which ACL2 processes mostly automatically. In addition to proving theorems about the translated LLVM code, ACL2 can also be used to execute test vectors at reasonable speed.

Note that you can see the intermediate form from clang with

```
clang -O4 -S -emit-llvm foo.c
```

Both Coq and the Hardin translator use OCAML [OCAM14] so we will have to learn that language.

Chapter 2

Theory

The proof of the Euclidean algorithm has been known since Euclid. We need to study an existing proof and use it to guide our use of Coq along the same lines, if possible. Some of the “obvious” natural language statements may require Coq lemmas.

From WikiProof [Wiki14a] we quote:

Let

$$a, b \in \mathbf{Z}$$

and $a \neq 0$ or $b \neq 0$.

The steps of the algorithm are:

1. Start with (a, b) such that $|a| \geq |b|$. If $b = 0$ then the task is complete and the GCD is a .
2. if $b \neq 0$ then you take the remainder r of a/b .
3. set $a \leftarrow b$, $b \leftarrow r$ (and thus $|a| \geq |b|$ again).
4. repeat these steps until $b = 0$

Thus the GCD of a and b is the value of the variable a at the end of the algorithm.

The proof is:

Suppose

$$a, b \in \mathbf{Z}$$

and $a \text{ or } b \neq 0$.

From the **division theorem**, $a = qb + r$ where $0 \leq r < |b|$

From **GCD with Remainder**, the GCD of a and b is also the GCD of b and r .

Therefore we may search instead for the $\text{gcd}(b, r)$.

Since $|r| \geq |b|$ and

$$b \in \mathbf{Z}$$

, we will reach $r = 0$ after finitely many steps.

At this point, $\gcd(r, 0) = r$ from **GCD with Zero**.

We quote the **Division Theorem** proof [Wiki14b]:

For every pair of integers a, b where $b \neq 0$, there exist unique integers q, r such that $a = qb + r$ and $0 \leq r < |b|$.

2.1 The Division Algorithm

From Judson [Juds15],

An Application of the Principle of Well-Ordering that we will use often is the division algorithm.

Theorem 2.9 Division Algorithm Let a and b be integers, with $b > 0$. Then there exists unique integers q and r such that

$$a = bq + r$$

where $0 \leq r < b$.

Proof

Let a and b be integers. If $b = ak$ for some integer k , we write $a|b$. An integer d is called a *common divisor* of a and b if $d|a$ and $d|b$. The *greatest common divisor* of integers a and b is a positive integer d such that d is a common divisor of a and b and if d' is any other common divisor of a and b , then $d'|d$. We write $d = \gcd(a, b)$; for example, $\gcd(24, 36) = 12$ and $\gcd(120, 102) = 6$. We say that two integers a and b are *relatively prime* if $\gcd(a, b) = 1$.

Theorem 2.10 Let a and b be nonzero integers. Then there exist integers r and s such that

$$\gcd(a, b) = ar + bs$$

Furthermore, the greatest common divisor of a and b is unique.

Proof

Collary 2.11 Let a and b be two integers that are relatively prime. Then there exist integers r and s such that

$$ar + bs = 1$$

The Euclidean Algorithm

Among other things, Theorem 2.10 allows us to compute the greatest common divisor of two integers.

Example 2.1.2 Let us compute the greatest common divisor of 945 and 2415. First observe

that

$$\begin{aligned} 2415 &= 945 \cdot 2 + 525 \\ 945 &= 525 \cdot 1 + 420 \\ 525 &= 420 \cdot 1 + 105 \\ 420 &= 105 \cdot 4 + 0 \end{aligned}$$

Reversing our steps, 105 divides 420, 105 divides 525, 105 divides 945, and 105 divides 2415. Hence, 105 divides both 945 and 2415. If d were another common divisor of 945 and 2415, then d would also have to divide 105. Therefore, $\gcd(945, 2415) = 105$.

If we work backward through the above sequence of equations, we can also obtain numbers r and s such that

$$945r + 2415s = 105$$

$$\begin{aligned} 105 &= 525 + (-1) \cdot 420 \\ 105 &= 525 + (-1) \cdot [945 + (-1) \cdot 525] \\ 105 &= 2 \cdot 525 + (-1) \cdot 945 \\ 105 &= 2 \cdot [2415 + (-2) \cdot 945] + (-1) \cdot 945 \\ 105 &= 2 \cdot 2415 + (-5) \cdot 945 \end{aligned}$$

So $r = -5$ and $s = 2$. Notice the r and s are not unique, since $r = 41$ and $s = -16$ would also work.

To compute $\gcd(a, b) = d$, we are using repeated divisions to obtain a decreasing sequence of positive integers $r_1 > r_2 > \dots > r_n = d$; that is

$$\begin{aligned} b &= aq_1 + r_1 \\ a &= r_1q_2 + r_2 \\ r_1 &= r_2q_3 + r_3 \\ &\vdots \\ r_{n-2} &= r_{n-1}q_n + r_n \\ r_{n-1} &= r_nq_{n+1} \end{aligned}$$

To find r and s such that $ar + bs = d$, we begin with the last equation and substitute results obtained from the previous equations:

$$\begin{aligned} d &= r_n \\ d &= r_{n-2} - r_{n-1}q_n \\ d &= r_{n-2} - q_n(r_{n-3} - q_{n-1}r_{n-2}) \\ d &= -q_nr_{n-3} + (1 + q_nq_{n-1})r_{n-2} \\ &\vdots \\ d &= ra + sb \end{aligned}$$

Chapter 3

Software Details

3.1 Installed Software

Install CLANG, LLVM

<http://llvm.org/releases/download.html>

Install OCAML

```
sudo apt-get install ocaml
```

An OCAML version of gcd would be written

```
let rec gcd a b = if b = 0 then a else gcd b (a mod b)
val gcd : int -> int -> int = <fun>
```


Chapter 4

Temporal Logic of Actions (TLA)

Sloppiness is easier than precision and rigor – Leslie Lamport[Lamp14a]

Leslie Lamport[Lamp14][Lamp16] on 21st Century Proofs.

A method of writing proofs is described that makes it harder to prove things that are not true. The method, based on hierarchical structuring, is simple and practical. The author's twenty years of experience writing such proofs is discussed.

Lamport points out that proofs need rigor and precision. Structure and Naming are important. Every step of the proof names the facts it uses.

Quoting from [Lamp16]:

Broadly speaking, a TLA+ proof is a collection of *claims*, arranged in a hierarchical structure which we describe below, where each claim has an *assertion* that is either *unjustified* or justified by a collection of *cited facts*. The purpose of TLAPS is to check the user-provided proofs of theorems, that is, to check that the hierarchy of claims indeed establishes the truth of the theorem if the claims were true, and then to check that the assertion of every justified claim indeed is implied by its cited facts. If a TLA+ theorem has a proof with no unjustified claims, then, as a result of checking the proof, TLAPS verifies the truth of the theorem.

4.1 The algorithm

The well-known Euclidean algorithm can be written in the PlusCal language as follows:

```
--algorithm Euclid {  
  variables x \in 1..M, y \in 1..N, x0 = x, y0 = y;  
  {  
    while (x # y) {
```

```

    if (x < y) { y := y - x; }
    else { x := x-y; }
  };
  assert x = GCD(x0, y0) /\ y = GCD(x0, y0)
}

```

The PlusCal translator translates this algorithm into a TLA+ specification that we could prove correct. However, in this tutorial, we shall write a somewhat simpler specification of Euclid's algorithm directly in TLA+.

Creating a new TLA+ module

In order to get the definitions of arithmetic operators (+, −, etc.), we shall make this specification *extend* the **Integers** standard module.

```

----- Module Euclid -----
EXTENDS Integers

```

Definitions

We shall then define the GCD of two integers. For that purpose, let us define the predicate “p divides q” as follows: p divides q iff there exists some integer d in the interval 1..q such that q is equal to p times d.

$$p \mid q == \exists d \in 1..q : q = p * d$$

We then define the set of divisors of an integer q as the sets of integers which both belong to the interval 1..q and divide q:

$$\text{Divisors}(q) == \{d \in 1..q : d \mid q\}$$

We define the maximum of a set S as one of the elements of this set which is greater than or equal to all the other elements:

$$\text{Maximum}(S) == \text{CHOOSE } x \in S : \forall y \in S : x \geq y$$

And finally, we define the GCD of two integers p and q to be the maximum of the intersection of Divisors(p) and Divisors(a):

$$\text{GCD}(p,q) == \text{Maximum}(\text{Divisors}(p) \cap \text{Divisors}(q))$$

For convenience, we shall also define the set of all positive integers as:

$$\text{Number} = \text{Nat} \setminus \{0\}$$

Constants and variables

We then define the two constants and two variables needed to describe the Euclidean algorithm, where M and N are the values whose GCD is to be computed:

```
CONSTANTS M, N
VARIABLES x, y
```

The specification

We define the initial state of the Euclidean algorithm as follows:

```
Init == (x = M) /\ (y = N)
```

In the Euclidean algorithm, two actions can be performed:

- set the value of y to $y - x$ if $x < y$
- set the value of x to $x - y$ if $x > y$

These actions are again written as a definition of **Next**, which specifies the next-state relation. In TLA+, a primed variable refers to its value at the next state of the algorithm.

```
Next == \/\ /\ x < y
        /\ y' = y - x
        /\ x' = x
        \/ /\ y < x
        /\ x' = x - y
        /\ y' = y
```

The specification of the algorithm asserts that the variables have the correct initial values and, in each execution step, either a **Next** action is performed or x and y keep the same values:

```
Spec == Init /\ [] [Next]_<<x,y>>
```

(For reasons that are irrelevant to this algorithm, TLA specifications always allow *stuttering steps* that leave all the variables unchanged.)

We want to prove that the algorithm always satisfies the following property:

```
ResultCorrect == (x = y) => x = GCD(M, N)
```

Hence we want to prove the following theorem named **Correctness**:

```
THEOREM Correctness == Spec => []ResultCorrect
```

Summary

```

----- Module Euclid -----
EXTENDS Integers

p | q == \E d \in 1..q : q = p * d
Divisors(q) == {d \in 1..q : d | q}
Maximum(S) == CHOOSE x \in S : \A y \in S : x >= y
GCD(p,q) == Maximum(Divisors(p) \cap Divisors(q))
Number == Nat \ {0}

CONSTANTS M, N
VARIABLES x, y

Init == (x = M) /\ (y = N)

Next == \ / /\ x < y
        /\ y' = y - x
        /\ x' = x
      \ / /\ y < x
        /\ x' = x-y
        /\ y' = y

Spec == Init /\ [] [Next]_<<x,y>>

ResultCorrect == (x = y) => x = GCD(M,N)

THEOREM Correctness == Spec => []ResultCorrect

```

4.2 A simple proof

The invariant

Intuitively, the theorem *Correctness* holds because the implementation guarantees the following *invariant*

```

InductiveInvariant == /\ x \in Number
                      /\ y \in Number
                      /\ GCD(x, y) = GCD(M, N)

```

That is, *InductiveInvariant* holds for the initial state (i.e., the state specified by *Init*) and is preserved by the next-state relation *[Next]*_{<<x,y>>}

Checking proofs

First we need to assume that constants *M* and *N* are not equal to zero

```
ASSUME NumberAssumption == M \in Number /\ N \in Number
```

Let us then prove that `InductiveInvariant` holds for the initial state.

```
THEOREM InitProperty == Init => InductiveInvariant
```

To check whether TLAPS can prove that theorem by itself, we declare its proof obvious.

```
THEOREM InitProperty == Init => InductiveInvariant
  OBVIOUS
```

We now ask TLAPS to prove that theorem. But TLAPS does not know how to prove the proof obligation corresponding to that proof. It prints that obligation and reports failures to three backends, Zenon, Isabelle, and SMT. The default behavior of TLAPS is to send obligations first to an SMT solver (by default CVC3), then if that fails to the automatic prover Zenon, then if Zenon fails to Isabelle (with the tactic “auto”).

Using facts and definitions

The obligation cannot be proved because TLAPS treats the symbols `Init` and `InductiveInvariant` as opaque identifiers unless it is explicitly instructed to expand their definitions using the directive `DEF`. The main purpose of this treatment of definitions is to make proof-checking tractable, because expanding definitions can arbitrarily increase the size of expressions. Explicit use of definitions is also a good hint to the (human) reader to look only at the listed definitions to understand a proof step. In the precise case, we can ask TLAPS to expand definitions of `Init` and `InductiveInvariant`, by replacing the proof `OBVIOUS` by the proof `BY DEF Init, InductiveInvariant`. In the obligations sent to the backends, the definitions of `Init` and `InductiveInvariant` have been expanded.

Unfortunately, none of the back-ends could prove that obligation. As with `definitions`, we have to specify which facts are *usable*. In this case, we have to make the fact `NumberAssumption` usable by changing the proof to

```
THEOREM InitProperty == Init => InductiveInvariant
  BY NumberAssumption DEF Init, InductiveInvariant
```

The general form of a BY proof is:

$$\text{BY } e_1, \dots, e_m \text{ DEF } d_1, \dots, d_n$$

which claims that the assertion follows by assuming $e_1, \dots, e_m$ and expanding the definitions $d_1, \dots, d_n$. It is the job of TLAPS to then check this claim, and also to check that the cited facts $e_1, \dots, e_m$ are indeed true.

Finally, SMT succeeds in proving that obligation.

```
----- Module Euclid -----
EXTENDS Integers
```

```

p | q == \E d \in 1..q : q = p * d
Divisors(q) == {d \in 1..q : d | q}
Maximum(S) == CHOOSE x \in S : \A y \in S : x >= y
GCD(p,q) == Maximum(Divisors(p) \cap Divisors(q))
Number == Nat \ {0}

CONSTANTS M, N
VARIABLES x, y

Init == (x = M) /\ (y = N)

Next == \ / /\ x < y
        /\ y' = y - x
        /\ x' = x
    \ / /\ y < x
        /\ x' = x-y
        /\ y' = y

Spec == Init /\ [] [Next]_<<x,y>>

ResultCorrect == (x = y) => x = GCD(M,N)

InductiveInvariant == /\ x \in Number
                      /\ y \in Number
                      /\ GCD(x, y) = GCD(M, N)

ASSUME NumberAssumption == M \in Number /\ N \in Number

THEOREM InitProperty == Init => InductiveInvariant
    BY NumberAssumption DEF Init, InductiveInvariant

THEOREM Correctness == Spec => []ResultCorrect

```

4.3 Divisibility Definition

In Shoup[Sho08] we find the divisibility definition.

Given the integers, a and b

$$a \text{ divides } b \implies az = b \text{ for some } z$$

so or all a, b , and c

$$a|a, \quad 1|a, \quad \text{and} \quad a|0$$

because $a \cdot 1 = a$, $1 \cdot a = a$, and $a \cdot 0 = 0$

$$0|a \iff a = 0$$

$$a|b \iff -a|b \iff a|-b$$

$$a|b \text{ and } a|c \implies a|(b+c)$$

$$a|b \text{ and } b|c \implies a|c$$

$$a|b \text{ and } b \neq 0 \implies 1 \leq |a| \leq |b|$$

$$az = b \neq 0 \text{ and } a \neq 0 \text{ and } z \neq 0 \implies |a| \geq 1 \text{ and } |z| \geq 1$$

$$a|b \text{ and } b|a \implies a = \pm b$$

proof:

$$a|b \implies |a| \leq |b|; b|a \implies |b| \leq |a|; \text{ therefore } |a| = |b| \implies a = \pm b$$

$$a|1 \iff a = \pm 1$$

Chapter 5

COQ proof of GCD

5.1 Basics of the Calculus of Constructions

Coquend[Coqu86][Wiki17] defines the Calculus of Constructions which can be considered an extension of the Curry-Howard Isomorphism. The components are

Terms

A *term* in the calculus of constructions is constructed using the following rules:

- **T** is a term (also called *Type*)
- **P** is a term (also called *Prop*, the type of all propositions)
- Variables $(x, y, \dots)$ are terms
- if **A** and **B** are terms, then so are
 - (A, B)
 - $(\lambda x : A, B)$
 - $(\forall x : A, B)$

The calculus of constructions has five kinds of objects:

1. *proofs*, which are terms whose types are *propositions*
2. *propositions*, which are also known as *small types*
3. *predicates*, which are functions that return propositions
4. *large types*, which are the types of predicates. **P** is an example of a large type)
5. **T** itself, which is the type of large types.

Judgements

The calculus of constructions allows proving **typing judgements**

$$x_1 : A_1, x_2 : A_2, \dots \vdash t : B$$

which can be read as the implication

if variables $x_1, x_2, \dots$, have types $A_1, A_2, \dots$, then term t has type B

The valid judgements for the calculus of constructions are derivable from a set of inference rules. In the following, we use Γ to mean a sequence of type assignments $x_1 : A_1, x_2 : A_2, \dots$, and we use $\mathbf{K}$ to mean either $\mathbf{P}$ or $\mathbf{T}$. We shall write $A : B : C$ to mean "A has type B, and B has type C". We shall write $B(x := N)$ to mean the result of substituting the term N for the variable x in the term B .

An inference rule is written in the form

$$\frac{\Gamma \vdash \mathbf{A} : \mathbf{B}}{\Gamma' \vdash \mathbf{C} : \mathbf{D}}$$

which means

if $\Gamma \vdash \mathbf{A} : \mathbf{B}$ is a valid judgement, then so is $\Gamma' \vdash \mathbf{C} : \mathbf{D}$

Inference Rules

In Frade[Frade08] we find:

(axiom)	$() \vdash s_1 : s_2$	if $(s_1, s_2) \in A$
(start)	$\frac{\Gamma \vdash A : s}{\Gamma, x : A \vdash x : A}$	if $x \notin \text{dom}(\Gamma)$
(weakening)	$\frac{\Gamma \vdash M : A \quad \Gamma \vdash B : s}{\Gamma, x : B \vdash M : A}$	if $x \notin \text{dom}(\Gamma)$
(product)	$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x : A \vdash B : s_2}{\Gamma \vdash (\prod x : A. B) : s_3}$	if $(s_1, s_2, s_3) \in \mathbb{R}$
(application)	$\frac{\Gamma \vdash M : (\prod x : A. B) \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[x := N]}$	
(abstraction)	$\frac{\Gamma, x : A \vdash M : B \quad \Gamma \vdash (\prod x : A. B) : s}{\Gamma \vdash \lambda x : A. M : (\prod x : A. B)}$	
(conversion)	$\frac{\Gamma \vdash M : A \quad \Gamma \vdash B : s}{\Gamma \vdash M : B}$	if $A =_\beta B$

Defining Logical Operators

$$\begin{aligned}
A \Rightarrow B &\equiv \quad \forall x : A.B && (x \notin B) \\
A \wedge B &\equiv \quad \forall C : P.(A \Rightarrow B \Rightarrow C) \Rightarrow C \\
A \vee B &\equiv \quad \forall C : P.(A \Rightarrow C) \Rightarrow (B \Rightarrow C) \Rightarrow C \\
\neg A &\equiv \quad \forall C : P.(A \Rightarrow C) \\
\exists x : A.B &\equiv \quad \forall C : P.(\forall x : A.(B \Rightarrow C)) \Rightarrow C
\end{aligned}$$

Defining Types

The basic data types used in computer science can be defined within the Calculus of Constructions:

Booleans

$$\forall A : P.A \Rightarrow A \Rightarrow A$$

Naturals

$$\forall A : P.(A \Rightarrow A) \Rightarrow (A \Rightarrow A)$$

Product $A \times B$

$$A \wedge B$$

Disjoint Union $A + B$

$$A \vee B$$

Note that Booleans and Naturals are defined in the same way as in Church encoding. However additional problems arise from propositional extensionality and proof irrelevance.

5.2 Why does COQ have Prop?

From a stackexchange post[[Stac17](#)] we find the question:

”Coq has a type **Prop** of proof irrelevant propositions which are discarded during extraction. What are the reasons for having this if we use Coq only for proofs? **Prop** is impredicative, however, Coq automatically infers universe indexes and we can use **Type(i)** instead everywhere. It seems **Prop** complicates everything a lot.”

Prop is very useful for program extraction because it allows us to delete parts of code that are useless. For example, to extract a sorting algorithm we would prove the statement “for every list l there is a list k such that k is ordered and k is a permutation of l ”. If we write this down in Coq and extract without using **Prop**, we will get:

1. “for all l there is a k ” which gives us a map **sort** which takes lists to lists,
2. “such that k is ordered” will give a function **verify** which runs through k and checks that it is sorted, and
3. “ k is a permutation of l will give a permutation **p1** which takes l to k . Note that **p1** is not just a mapping, but also the inverse mapping together with programs verifying that the two maps really are inverses.

While the extra stuff is not totally useless, in many applications we want to get ride of it and keep just **sort**. This can be accomplished if we use **Prop** to state “ k is ordered” and “ k is a permutation of l ”, but *not* “for all l there is k ”.

In general, a common way to extract code is to consider a statement of the form

$$\forall x : A. \exists y : B. \phi(x, y)$$

where x is input, y is output, and $\phi(x, y)$ explains what it means for y to be a correct output. (In the above example A and B are the types of lists and $\phi(l, k)$ is “ k is ordered and k is a permutation of l .”) if ϕ is in **Prop** then extraction gives a map $f : A \Rightarrow B$ such that $\phi(x, f(x))$ holds for all $x \in A$. If ϕ is in **Set** then we also get a function g such that $g(x)$ is the proof that $\phi(x, f(x))$ holds, for all $x \in A$. Often the proof is computationally useless and we prefer to get rid of it, especially when it is nested deeply inside some other statement. **Prop** gives use the possibility to do so.

There is a question whether we could avoid **Prop** altogether by automatically optimizing away “useless extracted code”. To some extent we can do that, for instance all code extracted from the negative fragment of logic (stuff build from the empty type, unit type, products) is useless as it just shuffles around the unit. But there are genuine design decisions one has to make when using **Prop**. Here is a simple example, where $\sum$ means that we are in **Type** and $\exists$ means we are in **Prop**. If we extract from

$$\prod_{n:N} \sum_{b:[0,1]} \sum_{k:N} n = 2 \cdot k + b$$

we will gat a program which decomposes n into its lowest bit b and the remaining bits k , i.e., it computes everything. If we extract from

$$\prod_{n:N} \sum_{b:[0,1]} \exists_{k:N} n = 2 \cdot k + b$$

then the program will only compute the lowest bit b . The machine cannot tell which is the correct one, the user has to tell it what he wants.

5.3 Source code of COQ GCD Proof

This is the proof of GCD[Coqu16a] in the COQ[Coqu16] sources:

```
Library Coq.ZArith.Znumtheory
```

```
Require Import ZArith_base.
Require Import ZArithRing.
Require Import Zcomplements.
Require Import Zdiv.
Require Import Wf_nat.
```

For compatibility reasons, this Open Scope isn't local as it should

```
Open Scope Z_scope.
```

This file contains some notions of number theory upon Z numbers:

```
  a divisibility predicate Z.divide
  a gcd predicate gcd
  Euclid algorithm euclid
  a relatively prime predicate rel_prime
  a prime predicate prime
  properties of the efficient Z.gcd function
```

```
Notation Zgcd := Z.gcd (compat "8.3").
Notation Zggcd := Z.ggcd (compat "8.3").
Notation Zggcd_gcd := Z.ggcd_gcd (compat "8.3").
Notation Zggcd_correct_divisors := Z.ggcd_correct_divisors (compat "8.3").
Notation Zgcd_divide_l := Z.gcd_divide_l (compat "8.3").
Notation Zgcd_divide_r := Z.gcd_divide_r (compat "8.3").
Notation Zgcd_greatest := Z.gcd_greatest (compat "8.3").
Notation Zgcd_nonneg := Z.gcd_nonneg (compat "8.3").
Notation Zggcd_opp := Z.ggcd_opp (compat "8.3").
```

The former specialized inductive predicate Z.divide is now a generic existential predicate.

```
Notation Zdivide := Z.divide (compat "8.3").
```

Its former constructor is now a pseudo-constructor.

```
Definition Zdivide_intro a b q (H:b=q*a) : Z.divide a b := ex_intro _ q H.
```

Results concerning divisibility

```
Notation Zdivide_refl := Z.divide_refl (compat "8.3").
Notation Zdivide_1_l := Z.divide_1_l (compat "8.3").
Notation Zdivide_0 := Z.divide_0_r (compat "8.3").
Notation Zmult_divide_compat_l := Z.mul_divide_mono_l (compat "8.3").
Notation Zmult_divide_compat_r := Z.mul_divide_mono_r (compat "8.3").
Notation Zdivide_plus_r := Z.divide_add_r (compat "8.3").
Notation Zdivide_minus_l := Z.divide_sub_r (compat "8.3").
Notation Zdivide_mult_l := Z.divide_mul_l (compat "8.3").
```

```

Notation Zdivide_mult_r := Z.divide_mul_r (compat "8.3").
Notation Zdivide_factor_r := Z.divide_factor_l (compat "8.3").
Notation Zdivide_factor_l := Z.divide_factor_r (compat "8.3").

Lemma Zdivide_opp_r a b : (a | b) -> (a | - b).

Lemma Zdivide_opp_r_rev a b : (a | - b) -> (a | b).

Lemma Zdivide_opp_l a b : (a | b) -> (- a | b).

Lemma Zdivide_opp_l_rev a b : (- a | b) -> (a | b).

Theorem Zdivide_Zabs_l a b : (Z.abs a | b) -> (a | b).

Theorem Zdivide_Zabs_inv_l a b : (a | b) -> (Z.abs a | b).

Hint Resolve Z.divide_refl Z.divide_1_l Z.divide_0_r: zarith.
Hint Resolve Z.mul_divide_mono_l Z.mul_divide_mono_r: zarith.
Hint Resolve Z.divide_add_r Zdivide_opp_r Zdivide_opp_r_rev Zdivide_opp_l
  Zdivide_opp_l_rev Z.divide_sub_r Z.divide_mul_l Z.divide_mul_r
  Z.divide_factor_l Z.divide_factor_r: zarith.

Auxiliary result.

Lemma Zmult_one x y : x >= 0 -> x * y = 1 -> x = 1.

Only 1 and -1 divide 1.

Notation Zdivide_1 := Z.divide_1_r (compat "8.3").

If a divides b and b divides a then a is b or -b.

Notation Zdivide_antisym := Z.divide_antisym (compat "8.3").
Notation Zdivide_trans := Z.divide_trans (compat "8.3").

If a divides b and b <> 0 then |a| <= |b|.

Lemma Zdivide_bounds a b : (a | b) -> b <> 0 -> Z.abs a <= Z.abs b.

Z.divide can be expressed using Z.modulo.

Lemma Zmod_divide : forall a b, b <> 0 -> a mod b = 0 -> (b | a).

Lemma Zdivide_mod : forall a b, (b | a) -> a mod b = 0.

Z.divide is hence decidable

Lemma Zdivide_dec a b : {(a | b)} + {~ (a | b)}.

Theorem Zdivide_Zdiv_eq a b : 0 < a -> (a | b) -> b = a * (b / a).

```

Theorem Zdivide_Zdiv_eq_2 a b c :
 $0 < a \rightarrow (a \mid b) \rightarrow (c * b) / a = c * (b / a).$

Theorem Zdivide_le: forall a b : Z,
 $0 \leq a \rightarrow 0 < b \rightarrow (a \mid b) \rightarrow a \leq b.$

Theorem Zdivide_Zdiv_lt_pos a b :
 $1 < a \rightarrow 0 < b \rightarrow (a \mid b) \rightarrow 0 < b / a < b.$

Lemma Zmod_div_mod n m a:
 $0 < n \rightarrow 0 < m \rightarrow (n \mid m) \rightarrow a \bmod n = (a \bmod m) \bmod n.$

Lemma Zmod_divide_minus a b c:
 $0 < b \rightarrow a \bmod b = c \rightarrow (b \mid a - c).$

Lemma Zdivide_mod_minus a b c:
 $0 \leq c < b \rightarrow (b \mid a - c) \rightarrow a \bmod b = c.$

Greatest common divisor (gcd).

There is no unicity of the gcd; hence we define the predicate Zis_gcd a b g expressing that g is a gcd of a

Inductive Zis_gcd (a b g:Z) : Prop :=
 Zis_gcd_intro :
 (g $\mid$ a) $\rightarrow$
 (g $\mid$ b) $\rightarrow$
 (forall x, (x $\mid$ a) $\rightarrow$ (x $\mid$ b) $\rightarrow$ (x $\mid$ g)) $\rightarrow$
 Zis_gcd a b g.

Trivial properties of gcd

Lemma Zis_gcd_sym : forall a b d, Zis_gcd a b d $\rightarrow$ Zis_gcd b a d.

Lemma Zis_gcd_0 : forall a, Zis_gcd a 0 a.

Lemma Zis_gcd_1 : forall a, Zis_gcd a 1 1.

Lemma Zis_gcd_refl : forall a, Zis_gcd a a a.

Lemma Zis_gcd_minus : forall a b d, Zis_gcd a (- b) d $\rightarrow$ Zis_gcd b a d.

Lemma Zis_gcd_opp : forall a b d, Zis_gcd a b d $\rightarrow$ Zis_gcd b a (- d).

Lemma Zis_gcd_0_abs a : Zis_gcd 0 a (Z.abs a).

Hint Resolve Zis_gcd_sym Zis_gcd_0 Zis_gcd_minus Zis_gcd_opp: zarith.

Theorem Zis_gcd_unique: forall a b c d : Z,
 $Zis_gcd\ a\ b\ c \rightarrow Zis_gcd\ a\ b\ d \rightarrow c = d \vee c = (-\ d).$

Extended Euclid algorithm.

Euclid's algorithm to compute the gcd mainly relies on the following property.

Lemma `Zis_gcd_for_euclid` :

forall a b d q:Z, $\text{Zis_gcd } b \ (a - q * b) \ d \rightarrow \text{Zis_gcd } a \ b \ d$.

Lemma `Zis_gcd_for_euclid2` :

forall b d q r:Z, $\text{Zis_gcd } r \ b \ d \rightarrow \text{Zis_gcd } b \ (b * q + r) \ d$.

We implement the extended version of Euclid's algorithm, i.e. the one computing Bezout's coefficients.

Section `extended_euclid_algorithm`.

Variables a b : Z.

The specification of Euclid's algorithm is the existence of u, v and d such that $ua+vb=d$ and $\text{gcd}(a,b)=d$.

Inductive `Euclid` : Set :=

`Euclid_intro` :

forall u v d:Z, $u * a + v * b = d \rightarrow \text{Zis_gcd } a \ b \ d \rightarrow \text{Euclid}$.

The recursive part of Euclid's algorithm uses well-founded recursion of non-negative integers. I

Lemma `euclid_rec` :

forall v3:Z,

$0 \leq v3 \rightarrow$

forall u1 u2 u3 v1 v2:Z,

$u1 * a + u2 * b = u3 \rightarrow$

$v1 * a + v2 * b = v3 \rightarrow$

$(\text{forall } d:Z, \text{Zis_gcd } u3 \ v3 \ d \rightarrow \text{Zis_gcd } a \ b \ d) \rightarrow \text{Euclid}$.

We get Euclid's algorithm by applying `euclid_rec` on $1,0,a,0,1,b$ when $b \geq 0$ and $1,0,a,0,-1,-b$ when $b < 0$.

Lemma `euclid` : `Euclid`.

End `extended_euclid_algorithm`.

Theorem `Zis_gcd_uniqueness_apart_sign` :

forall a b d d':Z, $\text{Zis_gcd } a \ b \ d \rightarrow \text{Zis_gcd } a \ b \ d' \rightarrow d = d' \vee d = -d'$.

Bezout's coefficients

Inductive `Bezout` (a b d:Z) : Prop :=

`Bezout_intro` : forall u v:Z, $u * a + v * b = d \rightarrow \text{Bezout } a \ b \ d$.

Existence of Bezout's coefficients for the gcd of a and b

Lemma `Zis_gcd_bezout` : forall a b d:Z, $\text{Zis_gcd } a \ b \ d \rightarrow \text{Bezout } a \ b \ d$.

gcd of ca and cb is c gcd(a,b).

```

Lemma Zis_gcd_mult :
  forall a b c d:Z, Zis_gcd a b d -> Zis_gcd (c * a) (c * b) (c * d).

Relative primality

Definition rel_prime (a b:Z) : Prop := Zis_gcd a b 1.

Bezout's theorem: a and b are relatively prime if and only if there exist u and v such that ua+vb = 1.

Lemma rel_prime_bezout : forall a b:Z, rel_prime a b -> Bezout a b 1.

Lemma bezout_rel_prime : forall a b:Z, Bezout a b 1 -> rel_prime a b.

Gauss's theorem: if a divides bc and if a and b are relatively prime, then a divides c.

Theorem Gauss : forall a b c:Z, (a | b * c) -> rel_prime a b -> (a | c).

If a is relatively prime to b and c, then it is to bc

Lemma rel_prime_mult :
  forall a b c:Z, rel_prime a b -> rel_prime a c -> rel_prime a (b * c).

Lemma rel_prime_cross_prod :
  forall a b c d:Z,
    rel_prime a b ->
    rel_prime c d -> b > 0 -> d > 0 -> a * d = b * c -> a = c /\ b = d.

After factorization by a gcd, the original numbers are relatively prime.

Lemma Zis_gcd_rel_prime :
  forall a b g:Z,
    b > 0 -> g >= 0 -> Zis_gcd a b g -> rel_prime (a / g) (b / g).

Theorem rel_prime_sym: forall a b, rel_prime a b -> rel_prime b a.

Theorem rel_prime_div: forall p q r,
  rel_prime p q -> (r | p) -> rel_prime r q.

Theorem rel_prime_1: forall n, rel_prime 1 n.

Theorem not_rel_prime_0: forall n, 1 < n -> ~ rel_prime 0 n.

Theorem rel_prime_mod: forall p q, 0 < q ->
  rel_prime p q -> rel_prime (p mod q) q.

Theorem rel_prime_mod_rev: forall p q, 0 < q ->
  rel_prime (p mod q) q -> rel_prime p q.

Theorem Zrel_prime_neq_mod_0: forall a b, 1 < b -> rel_prime a b -> a mod b <> 0.

```

Primality

```
Inductive prime (p:Z) : Prop :=
  prime_intro :
    1 < p -> (forall n:Z, 1 <= n < p -> rel_prime n p) -> prime p.
```

The sole divisors of a prime number p are -1 , 1 , p and $-p$.

```
Lemma prime_divisors :
  forall p:Z,
    prime p -> forall a:Z, (a | p) -> a = -1 \/ a = 1 \/ a = p \/ a = - p.
```

A prime number is relatively prime with any number it does not divide

```
Lemma prime_rel_prime :
  forall p:Z, prime p -> forall a:Z, ~ (p | a) -> rel_prime p a.
```

Hint Resolve prime_rel_prime: zarith.

As a consequence, a prime number is relatively prime with smaller numbers

```
Theorem rel_prime_le_prime:
  forall a p, prime p -> 1 <= a < p -> rel_prime a p.
```

If a prime p divides ab then it divides either a or b

```
Lemma prime_mult :
  forall p:Z, prime p -> forall a b:Z, (p | a * b) -> (p | a) \/ (p | b).
```

Lemma not_prime_0: $\sim$ prime 0.

Lemma not_prime_1: $\sim$ prime 1.

Lemma prime_2: prime 2.

Theorem prime_3: prime 3.

Theorem prime_ge_2 p : prime p -> $2 \leq p$.

Definition prime' p := $1 < p \wedge (\text{forall } n, 1 < n < p \rightarrow \sim (n | p))$.

Lemma Z_0_1_more x : $0 \leq x \rightarrow x = 0 \vee x = 1 \vee 1 < x$.

Theorem prime_alt p : prime' p $\leftrightarrow$ prime p.

Theorem square_not_prime: forall a, $\sim$ prime (a * a).

```
Theorem prime_div_prime: forall p q,
  prime p -> prime q -> (p | q) -> p = q.
```

we now prove that $Z.\text{gcd}$ is indeed a gcd in the sense of Zis_gcd .

Notation $Zgcd_is_pos := Z.\text{gcd_nonneg}$ (compat "8.3").

Lemma $Zgcd_is_gcd$: forall a b, Zis_gcd a b $(Z.\text{gcd}$ a b).

Theorem $Zgcd_spec$: forall x y : Z, $\{z : Z \mid Zis_gcd$ x y z $\wedge 0 \leq z\}$.

Theorem $Zdivide_Zgcd$: forall p q r : Z,
 $(p \mid q) \rightarrow (p \mid r) \rightarrow (p \mid Z.\text{gcd}$ q r).

Theorem Zis_gcd_gcd : forall a b c : Z,
 $0 \leq c \rightarrow Zis_gcd$ a b c $\rightarrow Z.\text{gcd}$ a b = c.

Notation $Zgcd_inv_0_l := Z.\text{gcd_eq_0_l}$ (compat "8.3").

Notation $Zgcd_inv_0_r := Z.\text{gcd_eq_0_r}$ (compat "8.3").

Theorem $Zgcd_div_swap0$: forall a b : Z,
 $0 < Z.\text{gcd}$ a b $\rightarrow$
 $0 < b \rightarrow$
 $(a / Z.\text{gcd}$ a b) * b = a * (b / Z.\text{gcd} a b).

Theorem $Zgcd_div_swap$: forall a b c : Z,
 $0 < Z.\text{gcd}$ a b $\rightarrow$
 $0 < b \rightarrow$
 $(c * a) / Z.\text{gcd}$ a b * b = c * a * (b / Z.\text{gcd} a b).

Notation $Zgcd_comm := Z.\text{gcd_comm}$ (compat "8.3").

Lemma $Zgcd_ass$ a b c : $Z.\text{gcd}$ (Z.\text{gcd} a b) c = $Z.\text{gcd}$ a (Z.\text{gcd} b c).

Notation $Zgcd_Zabs := Z.\text{gcd_abs_l}$ (compat "8.3").

Notation $Zgcd_0 := Z.\text{gcd_0_r}$ (compat "8.3").

Notation $Zgcd_1 := Z.\text{gcd_1_r}$ (compat "8.3").

Hint Resolve $Z.\text{gcd_0_r}$ $Z.\text{gcd_1_r}$: zarith.

Theorem $Zgcd_1_rel_prime$: forall a b,
 $Z.\text{gcd}$ a b = 1 $\leftrightarrow rel_prime$ a b.

Definition rel_prime_dec : forall a b,
 $\{ rel_prime$ a b $\} + \{ \sim rel_prime$ a b $\}$.

Definition $prime_dec_aux$:
forall p m,
 $\{ forall$ n, $1 < n < m \rightarrow rel_prime$ n p $\} +$
 $\{ exists$ n, $1 < n < m \wedge \sim rel_prime$ n p $\}$.

Definition $prime_dec$: forall p, $\{ prime$ p $\} + \{ \sim prime$ p $\}$.

Theorem not_prime_divide:

forall p, 1 < p -> ~ prime p -> exists n, 1 < n < p /\ (n | p).

Chapter 6

LEAN proof of GCD

This is the proof of GCD[Avig14] in the LEAN[Avig16] sources:

```
/-
Copyright (c) 2014 Jeremy Avigad. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Jeremy Avigad, Leonardo de Moura

Definitions and properties of gcd, lcm, and coprime.
-/
import .div
open eq.ops well_founded decidable prod

namespace nat

/- gcd -/

private definition pair_nat.lt : nat → nat → nat → nat → Prop := measure pr
private definition pair_nat.lt.wf : well_founded pair_nat.lt :=
intro_k (measure.wf pr) 20 -- we use intro_k to be able to execute gcd efficiently in the kernel

local attribute pair_nat.lt.wf [instance] -- instance will not be saved in .olean
local infixl ' ' :50 := pair_nat.lt

private definition gcd.lt.dec (x y : nat) : (succ y, x % succ y) → (x, succ y) :=
!mod_lt (succ_pos y)

definition gcd.F : (p : nat → nat), (p : nat → nat, p p nat) → nat
| (x, 0) f := x
| (x, succ y) f := f (succ y, x % succ y) !gcd.lt.dec

definition gcd (x y : nat) := fix gcd.F (x, y)
```

```

theorem gcd_zero_right [simp] (x : nat) : gcd x 0 = x := rfl

theorem gcd_succ [simp] (x y : nat) : gcd x (succ y) = gcd (succ y) (x % succ y) :=
  well_founded.fix_eq gcd.F (x, succ y)

theorem gcd_one_right (n : ) : gcd n 1 = 1 :=
  calc gcd n 1 = gcd 1 (n % 1) : gcd_succ
    ... = gcd 1 0 : mod_one

theorem gcd_def (x : ) : (y : ), gcd x y = if y = 0 then x else gcd y (x % y)
| 0 := !gcd_zero_right
| (succ y) := !gcd_succ (if_neg !succ_ne_zero)

theorem gcd_self : (n : ), gcd n n = n
| 0 := rfl
| (succ n) := calc
  gcd (succ n) (succ n) = gcd (succ n) (succ n % succ n) : gcd_succ
  ... = gcd (succ n) 0 : mod_self

theorem gcd_zero_left : (n : ), gcd 0 n = n
| 0 := rfl
| (succ n) := calc
  gcd 0 (succ n) = gcd (succ n) (0 % succ n) : gcd_succ
  ... = gcd (succ n) 0 : zero_mod

theorem gcd_of_pos (m : ) {n : } (H : n > 0) : gcd m n = gcd n (m % n) :=
  gcd_def m n if_neg (ne_zero_of_pos H)

theorem gcd_rec (m n : ) : gcd m n = gcd n (m % n) :=
  by_cases_zero_pos n
  (calc
    m = gcd 0 m : gcd_zero_left
    ... = gcd 0 (m % 0) : mod_zero)
  (take n, assume H : 0 < n, gcd_of_pos m H)

theorem gcd.induction {P : Prop}
  (m n : )
  (H0 : m, P m 0)
  (H1 : m n, 0 < n → P n (m % n) → P m n) :
  P m n :=
  induction (m, n) (prod.rec (m, nat.rec (IH, H0 m)
    ( n v (IH : p, p (m, succ n) → P (pr p) (pr p)),
    H1 m (succ n) !succ_pos (IH _ !gcd.lt.dec))))

theorem gcd_dvd (m n : ) : (gcd m n ∣ m) (gcd m n ∣ n) :=
  gcd.induction m n
  (take m, and.intro (!one_mul !dvd_mul_left) !dvd_zero)
  (take m n (npos : 0 < n), and.rec
    (assume (IH : gcd n (m % n) ∣ n) (IH : gcd n (m % n) ∣ (m % n)),

```

```

have H : (gcd n (m % n) (m / n * n + m % n)), from
  dvd_add (dvd.trans IH !dvd_mul_left) IH,
have H1 : (gcd n (m % n) m), from !eq_div_mul_add_mod H,
show (gcd m n m) (gcd m n n), from !gcd_rec (and.intro H1 IH))

theorem gcd_dvd_left (m n : ) : gcd m n m := and.left !gcd_dvd

theorem gcd_dvd_right (m n : ) : gcd m n n := and.right !gcd_dvd

theorem dvd_gcd {m n k : } : k m k n k gcd m n :=
gcd.induction m n (take m, imp.intro)
  (take m n (npos : n > 0)
    (IH : k n k m % n k gcd n (m % n))
    (H1 : k m) (H2 : k n),
    have H3 : k m / n * n + m % n, from !eq_div_mul_add_mod H1,
    have H4 : k m % n, from nat.dvd_of_dvd_add_left H3 (dvd.trans H2 !dvd_mul_left),
    !gcd_rec IH H2 H4)

theorem gcd.comm (m n : ) : gcd m n = gcd n m :=
dvd.antisymm
  (dvd_gcd !gcd_dvd_right !gcd_dvd_left)
  (dvd_gcd !gcd_dvd_right !gcd_dvd_left)

theorem gcd.assoc (m n k : ) : gcd (gcd m n) k = gcd m (gcd n k) :=
dvd.antisymm
  (dvd_gcd
    (dvd.trans !gcd_dvd_left !gcd_dvd_left)
    (dvd_gcd (dvd.trans !gcd_dvd_left !gcd_dvd_right) !gcd_dvd_right))
  (dvd_gcd
    (dvd_gcd !gcd_dvd_left (dvd.trans !gcd_dvd_right !gcd_dvd_left))
    (dvd.trans !gcd_dvd_right !gcd_dvd_right))

theorem gcd_one_left (m : ) : gcd 1 m = 1 :=
!gcd.comm !gcd_one_right

theorem gcd_mul_left (m n k : ) : gcd (m * n) (m * k) = m * gcd n k :=
gcd.induction n k
  (take n, calc gcd (m * n) (m * 0) = gcd (m * n) 0 : mul_zero)
  (take n k,
    assume H : 0 < k,
    assume IH : gcd (m * k) (m * (n % k)) = m * gcd k (n % k),
    calc
      gcd (m * n) (m * k) = gcd (m * k) (m * n % (m * k)) : !gcd_rec
      ... = gcd (m * k) (m * (n % k)) : mul_mod_mul_left
      ... = m * gcd k (n % k) : IH
      ... = m * gcd n k : !gcd_rec)

theorem gcd_mul_right (m n k : ) : gcd (m * n) (k * n) = gcd m k * n :=
calc
  gcd (m * n) (k * n) = gcd (n * m) (k * n) : mul.comm

```

```

... = gcd (n * m) (n * k) : mul.comm
... = n * gcd m k          : gcd_mul_left
... = gcd m k * n          : mul.comm

theorem gcd_pos_of_pos_left {m : } (n : ) (mpos : m > 0) : gcd m n > 0 :=
pos_of_dvd_of_pos !gcd_dvd_left mpos

theorem gcd_pos_of_pos_right (m : ) {n : } (npos : n > 0) : gcd m n > 0 :=
pos_of_dvd_of_pos !gcd_dvd_right npos

theorem eq_zero_of_gcd_eq_zero_left {m n : } (H : gcd m n = 0) : m = 0 :=
or.elim (eq_zero_or_pos m)
  (assume H1, H1)
  (assume H1 : m > 0, absurd H (ne_of_lt (!gcd_pos_of_pos_left H1)))

theorem eq_zero_of_gcd_eq_zero_right {m n : } (H : gcd m n = 0) : n = 0 :=
eq_zero_of_gcd_eq_zero_left (!gcd.comm H)

theorem gcd_div {m n k : } (H1 : k ∣ m) (H2 : k ∣ n) :
gcd (m / k) (n / k) = gcd m n / k :=
or.elim (eq_zero_or_pos k)
  (assume H3 : k = 0, by subst k; rewrite *nat.div_zero)
  (assume H3 : k > 0, (nat.div_eq_of_eq_mul_left H3 (calc
    gcd m n = gcd m (n / k * k)          : nat.div_mul_cancel H2
    ... = gcd (m / k * k) (n / k * k) : nat.div_mul_cancel H1
    ... = gcd (m / k) (n / k) * k      : gcd_mul_right)))

theorem gcd_dvd_gcd_mul_left (m n k : ) : gcd m n ∣ gcd (k * m) n :=
dvd_gcd (dvd.trans !gcd_dvd_left !dvd_mul_left) !gcd_dvd_right

theorem gcd_dvd_gcd_mul_right (m n k : ) : gcd m n ∣ gcd (m * k) n :=
!mul.comm !gcd_dvd_gcd_mul_left

theorem gcd_dvd_gcd_mul_left_right (m n k : ) : gcd m n ∣ gcd m (k * n) :=
dvd_gcd !gcd_dvd_left (dvd.trans !gcd_dvd_right !dvd_mul_left)

theorem gcd_dvd_gcd_mul_right_right (m n k : ) : gcd m n ∣ gcd m (n * k) :=
!mul.comm !gcd_dvd_gcd_mul_left_right

/- lcm -/

definition lcm (m n : ) : := m * n / (gcd m n)

theorem lcm.comm (m n : ) : lcm m n = lcm n m :=
calc
  lcm m n = m * n / gcd m n : rfl
  ... = n * m / gcd m n : mul.comm
  ... = n * m / gcd n m : gcd.comm
  ... = lcm n m          : rfl

```

```

theorem lcm_zero_left (m : ℕ) : lcm 0 m = 0 :=
calc
  lcm 0 m = 0 * m / gcd 0 m : rfl
  ... = 0 / gcd 0 m      : zero_mul
  ... = 0                  : nat.zero_div

theorem lcm_zero_right (m : ℕ) : lcm m 0 = 0 := !lcm.comm !lcm_zero_left

theorem lcm_one_left (m : ℕ) : lcm 1 m = m :=
calc
  lcm 1 m = 1 * m / gcd 1 m : rfl
  ... = m / gcd 1 m      : one_mul
  ... = m / 1             : gcd_one_left
  ... = m                  : nat.div_one

theorem lcm_one_right (m : ℕ) : lcm m 1 = m := !lcm.comm !lcm_one_left

theorem lcm_self (m : ℕ) : lcm m m = m :=
have H : m * m / m = m, from
  by_cases_zero_pos m !nat.div_zero (take m, assume H1 : m > 0, !nat.mul_div_cancel H1),
calc
  lcm m m = m * m / gcd m m : rfl
  ... = m * m / m           : gcd_self
  ... = m                    : H

theorem dvd_lcm_left (m n : ℕ) : m ∣ lcm m n :=
have H : lcm m n = m * (n / gcd m n), from nat.mul_div_assoc _ !gcd_dvd_right,
dvd.intro H

theorem dvd_lcm_right (m n : ℕ) : n ∣ lcm m n :=
!lcm.comm !dvd_lcm_left

theorem gcd_mul_lcm (m n : ℕ) : gcd m n * lcm m n = m * n :=
eq.symm (nat.eq_mul_of_div_eq_right (dvd.trans !gcd_dvd_left !dvd_mul_right) rfl)

theorem lcm_dvd {m n k : ℕ} (H1 : m ∣ k) (H2 : n ∣ k) : lcm m n ∣ k :=
or.elim (eq_zero_or_pos k)
  (assume kzero : k = 0, !kzero !dvd_zero)
  (assume kpos : k > 0,
    have mpos : m > 0, from pos_of_dvd_of_pos H1 kpos,
    have npos : n > 0, from pos_of_dvd_of_pos H2 kpos,
    have gcd_pos : gcd m n > 0, from !gcd_pos_of_pos_left mpos,
    obtain p (km : k = m * p), from exists_eq_mul_right_of_dvd H1,
    obtain q (kn : k = n * q), from exists_eq_mul_right_of_dvd H2,
    have ppos : p > 0, from pos_of_mul_pos_left (km kpos),
    have qpos : q > 0, from pos_of_mul_pos_left (kn kpos),
    have H3 : p * q * (m * n * gcd p q) = p * q * (gcd m n * k), from
    calc
      p * q * (m * n * gcd p q)
        = m * p * (n * q * gcd p q) : by rewrite [*mul.assoc, *mul.left_comm q,

```

```

... = k * (k * gcd p q)                : by rewrite [-kn, -km]
... = k * gcd (k * p) (k * q)          : by rewrite gcd_mul_left
... = k * gcd (n * q * p) (m * p * q) : by rewrite [-kn, -km]
... = k * (gcd n m * (p * q))          : by rewrite [*mul.assoc, mul.comm q, gcd_mul_right]
... = p * q * (gcd m n * k)            : by rewrite [mul.comm, mul.comm (gcd n m), gcd.comm m,
                                              *mul.assoc],

have H4 : m * n * gcd p q = gcd m n * k,
  from !eq_of_mul_eq_mul_left (mul_pos ppos qpos) H3,
have H5 : gcd m n * (lcm m n * gcd p q) = gcd m n * k,
  from !mul.assoc !gcd_mul_lcm H4,
have H6 : lcm m n * gcd p q = k,
  from !eq_of_mul_eq_mul_left gcd_pos H5,
dvd.intro H6)

theorem lcm.assoc (m n k : ) : lcm (lcm m n) k = lcm m (lcm n k) :=
dvd.antisymm
  (lcm_dvd
    (lcm_dvd !dvd_lcm_left (dvd.trans !dvd_lcm_left !dvd_lcm_right))
    (dvd.trans !dvd_lcm_right !dvd_lcm_right))
  (lcm_dvd
    (dvd.trans !dvd_lcm_left !dvd_lcm_left)
    (lcm_dvd (dvd.trans !dvd_lcm_right !dvd_lcm_left) !dvd_lcm_right))

/- coprime -/

definition coprime [reducible] (m n : ) : Prop := gcd m n = 1

lemma gcd_eq_one_of_coprime {m n : } : coprime m n → gcd m n = 1 :=
h, h

theorem coprime_swap {m n : } (H : coprime n m) : coprime m n :=
!gcd.comm H

theorem dvd_of_coprime_of_dvd_mul_right {m n k : } (H1 : coprime k n) (H2 : k ∣ m * n) : k ∣ m :=
have H3 : gcd (m * k) (m * n) = m, from
  calc
    gcd (m * k) (m * n) = m * gcd k n : gcd_mul_left
    ... = m * 1          : H1
    ... = m              : mul_one,
have H4 : (k ∣ gcd (m * k) (m * n)), from dvd_gcd !dvd_mul_left H2,
H3 H4

theorem dvd_of_coprime_of_dvd_mul_left {m n k : } (H1 : coprime k m) (H2 : k ∣ m * n) : k ∣ n :=
dvd_of_coprime_of_dvd_mul_right H1 (!mul.comm H2)

theorem gcd_mul_left_cancel_of_coprime {k : } (m : ) {n : } (H : coprime k n) :
gcd (k * m) n = gcd m n :=
have H1 : coprime (gcd (k * m) n) k, from
  calc

```

```

gcd (gcd (k * m) n) k
  = gcd (k * gcd 1 m) n : by rewrite [-gcd_mul_left, mul_one, gcd.comm, gcd.assoc]
... = 1                  : by rewrite [gcd_one_left, mul_one, coprime at H, H],
dvd.antisymm
  (dvd_gcd (dvd_of_coprime_of_dvd_mul_left H1 !gcd_dvd_left) !gcd_dvd_right)
  (dvd_gcd (dvd.trans !gcd_dvd_left !dvd_mul_left) !gcd_dvd_right)

theorem gcd_mul_right_cancel_of_coprime (m : ℕ) {k n : ℕ} (H : coprime k n) :
  gcd (m * k) n = gcd m n :=
!mul.comm !gcd_mul_left_cancel_of_coprime H

theorem gcd_mul_left_cancel_of_coprime_right {k m : ℕ} (n : ℕ) (H : coprime k m) :
  gcd m (k * n) = gcd m n :=
!gcd.comm !gcd.comm !gcd_mul_left_cancel_of_coprime H

theorem gcd_mul_right_cancel_of_coprime_right {k m : ℕ} (n : ℕ) (H : coprime k m) :
  gcd m (n * k) = gcd m n :=
!gcd.comm !gcd.comm !gcd_mul_right_cancel_of_coprime H

theorem coprime_div_gcd_div_gcd {m n : ℕ} (H : gcd m n > 0) :
  coprime (m / gcd m n) (n / gcd m n) :=
calc
  gcd (m / gcd m n) (n / gcd m n) = gcd m n / gcd m n : gcd_div !gcd_dvd_left !gcd_dvd_right
  ... = 1 : nat.div_self H

theorem not_coprime_of_dvd_of_dvd {m n d : ℕ} (dgt1 : d > 1) (Hm : d ∣ m) (Hn : d ∣ n) :
  coprime m n :=
assume co : coprime m n,
have d ∣ gcd m n, from dvd_gcd Hm Hn,
have d = 1, by rewrite [coprime at co, co at this]; apply this,
have d = 1, from le_of_dvd dec_trivial this,
show false, from not_lt_of_ge 'd = 1' 'd > 1'

theorem exists_coprime {m n : ℕ} (H : gcd m n > 0) :
  exists m' n', coprime m' n' m = m' * gcd m n n = n' * gcd m n :=
have H1 : m = (m / gcd m n) * gcd m n, from (nat.div_mul_cancel !gcd_dvd_left),
have H2 : n = (n / gcd m n) * gcd m n, from (nat.div_mul_cancel !gcd_dvd_right),
exists.intro _ (exists.intro _ (and.intro (coprime_div_gcd_div_gcd H) (and.intro H1 H2)))

theorem coprime_mul {m n k : ℕ} (H1 : coprime m k) (H2 : coprime n k) : coprime (m * n) k :=
calc
  gcd (m * n) k = gcd n k : !gcd_mul_left_cancel_of_coprime H1
  ... = 1          : H2

theorem coprime_mul_right {k m n : ℕ} (H1 : coprime k m) (H2 : coprime k n) : coprime k (m * n) :=
coprime_swap (coprime_mul (coprime_swap H1) (coprime_swap H2))

theorem coprime_of_coprime_mul_left {k m n : ℕ} (H : coprime (k * m) n) : coprime m n :=
have H1 : (gcd m n ∣ gcd (k * m) n), from !gcd_dvd_gcd_mul_left,
eq_one_of_dvd_one (H H1)

```

```

theorem coprime_of_coprime_mul_right {k m n : } (H : coprime (m * k) n) : coprime m n :=
coprime_of_coprime_mul_left (!mul.comm H)

theorem coprime_of_coprime_mul_left_right {k m n : } (H : coprime m (k * n)) : coprime m n :=
coprime_swap (coprime_of_coprime_mul_left (coprime_swap H))

theorem coprime_of_coprime_mul_right_right {k m n : } (H : coprime m (n * k)) : coprime m n :=
coprime_of_coprime_mul_left_right (!mul.comm H)

theorem coprime_one_left : n, coprime 1 n :=
n, !gcd_one_left

theorem coprime_one_right : n, coprime n 1 :=
n, !gcd_one_right

theorem exists_eq_prod_and_dvd_and_dvd {m n k : nat} (H : k m * n) :
  m' n', k = m' * n' m' m n' n :=
or.elim (eq_zero_or_pos (gcd k m))
  (assume H1 : gcd k m = 0,
    have H2 : k = 0, from eq_zero_of_gcd_eq_zero_left H1,
    have H3 : m = 0, from eq_zero_of_gcd_eq_zero_right H1,
    have H4 : k = 0 * n, from H2 !zero_mul,
    have H5 : 0 m, from H3 !dvd.refl,
    have H6 : n n, from !dvd.refl,
    exists.intro _ (exists.intro _ (and.intro H4 (and.intro H5 H6))))
  (assume H1 : gcd k m > 0,
    have H2 : gcd k m k, from !gcd_dvd_left,
    have H3 : k / gcd k m (m * n) / gcd k m, from nat.div_dvd_div H2 H,
    have H4 : (m * n) / gcd k m = (m / gcd k m) * n, from
      calc
        m * n / gcd k m = n * m / gcd k m : mul.comm
        ... = n * (m / gcd k m) : !nat.mul_div_assoc !gcd_dvd_right
        ... = m / gcd k m * n : mul.comm,
    have H5 : k / gcd k m (m / gcd k m) * n, from H4 H3,
    have H6 : coprime (k / gcd k m) (m / gcd k m), from coprime_div_gcd_div_gcd H1,
    have H7 : k / gcd k m n, from dvd_of_coprime_of_dvd_mul_left H6 H5,
    have H8 : k = gcd k m * (k / gcd k m), from (nat.mul_div_cancel' H2),
    exists.intro _ (exists.intro _ (and.intro H8 (and.intro !gcd_dvd_right H7))))
end nat

```

Chapter 7

Formal Pre- and Post-conditions

In Boldo[[Bold11](#)] we find an effort to verify floating point software using preconditions, postconditions, and assertions. Quoting:

“These conjectures can be described formally by annotations as follows.

```
/*@ requires \abs(x) <= 0x1p-5;
   @ ensures \abs(\result - \cos(x)) <= 0x1p-23;
   @*/
float my_cosine(float x) {
    //@ assert \abs(1.0 - x*x*0.5 - \cos(x)) < 0x1p-24;
    return 1.0f - x * x * 0.5f;
}
```

The *precondition*, introduced by **requires**, states that we expect argument x in the interval $[-1/32; 1/32]$. The *postcondition*, introduced by **ensures**, states that the distance between the value returned by the function, denoted by the keyword **\result**, and the model of the program, which is here the true mathematical cosine function denoted by **\cos** in ACSL, is not greater than 2^{-23} . It is important to notice that in annotations the operators like $+$ or $*$ denote operations on real numbers and not on floating-point numbers. In particular, there is no rounding error and no overflow in annotations, unlike in the early Leavens’ proposal. The C variables of type **float**, like **x** and **\result** in this example, are interpreted as the real number they represent. Thus, the last annotation, given as an assertion inside the code, is a way to make explicit the reasoning we made above, making the total error the sum of the method error and the rounding error: it states that the method error is less than 2^{-24} . Again, it is thanks to the choice of having exact operations in the annotations that we are able to state a property of the method error.”

In Boldo[[Bold07](#), [Bold07a](#)] we find ‘search in an array’ annotated:

```
/*@ requires \valid_range(t,0,n-1)
   @ ensures
   @   (0 <= \result < n => t[\result] == v) &&
```

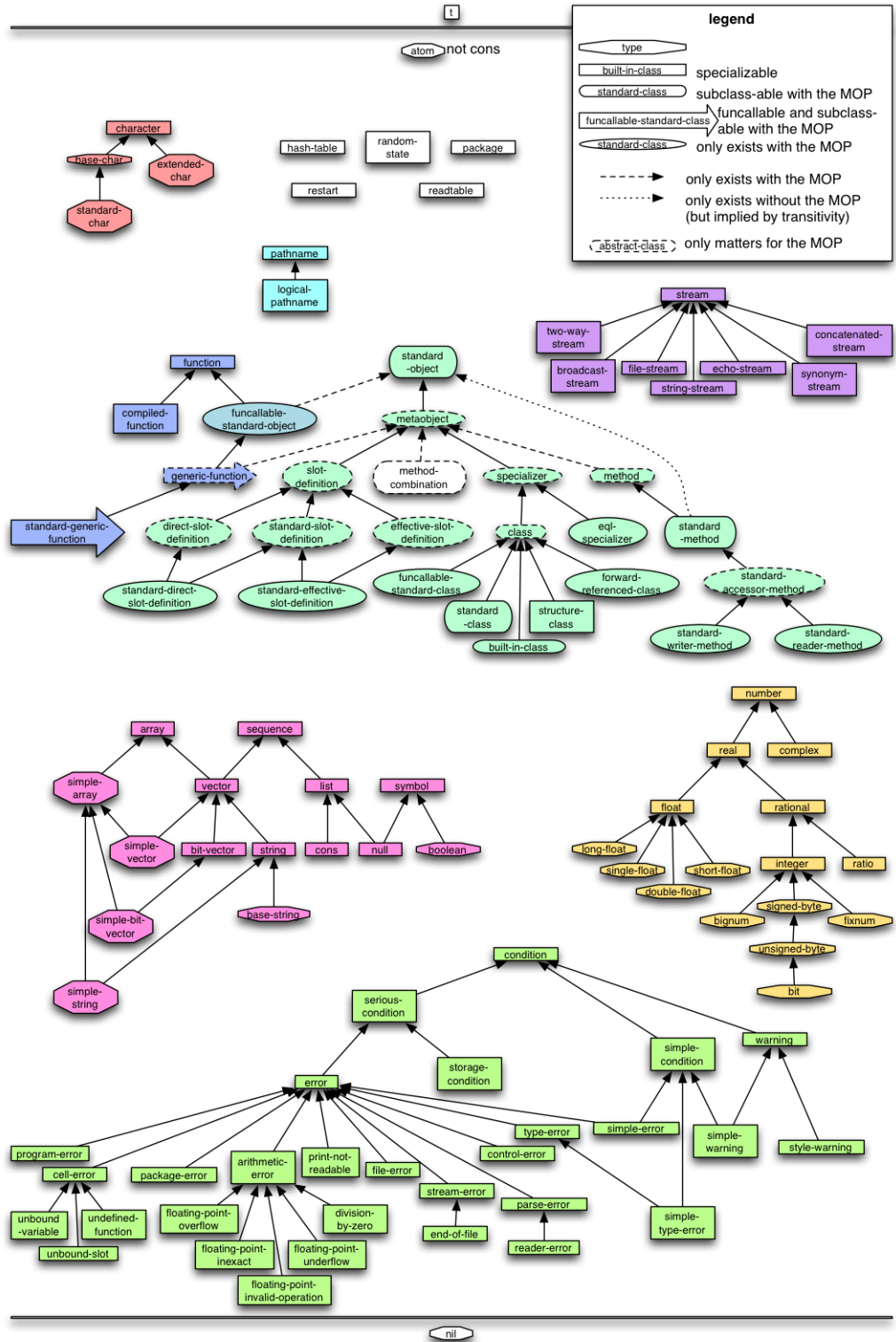
```
@ (\result == n =>
@   \forall int i; 0 <= i < n => t[i] != v) */
int index(int t[], int n, int v) {
    int i = 0;
    /*@ invariant 0 <= i &&
    @   \forall int k; 0 <= k < i => t[k] != v
    @ variant n - i */
    while (i < n) {
        if (t[i] == v) break;
        i++;
    }
    return i;
}
```

Chapter 8

Types and Signatures

We need to start from a base of the existing types in Common Lisp, eventually providing Axiom combinations or specializations. Common Lisp has these standard type specifier symbols.

Common Lisp Type Hierarchy[\[Pfei12\]](#)



Axiom adds these types:

- `Command = String`

Chapter 9

COQ nat vs Axiom NNI

COQ's nat domain includes a proof of GCD.

We would like to show an isomorphism between types in Coq and types in Axiom. Having such an isomorphism will make lemmas available and simplify future proofs.

Note that Coq's `nat` domain stops at 0 (a symbolic 0) as does Axiom's NNI. The Axiom interpreter will promote a subtraction to Integer whereas Coq will not.

COQ's nat domain[`COQnat`] is

Library Coq.Init.Nat

```
Require Import Notations Logic Datatypes.
```

```
Local Open Scope nat_scope.
```

Peano natural numbers, definitions of operations

This file is meant to be used as a whole module, without importing it, leading to qualified definitions (e.g. `Nat.pred`)

```
Definition t := nat.
```

Constants

```
Definition zero := 0.
```

```
Definition one := 1.
```

```
Definition two := 2.
```

Basic operations

```
Definition succ := S.
```

```

Definition pred n :=
  match n with
  | 0 => n
  | S u => u
  end.

```

```

Fixpoint add n m :=
  match n with
  | 0 => m
  | S p => S (p + m)
  end

```

```

where "n + m" := (add n m) : nat_scope.

```

```

Definition double n := n + n.

```

```

Fixpoint mul n m :=
  match n with
  | 0 => 0
  | S p => m + p * m
  end

```

```

where "n * m" := (mul n m) : nat_scope.

```

Note that Axiom's NNI domain will be automatically promoted to Integer when the subtraction result is negative. Coq returns O when this occurs.

Truncated subtraction: $n-m$ is 0 if $n \leq m$

```

Fixpoint sub n m :=
  match n, m with
  | S k, S l => k - l
  | _, _ => n
  end

```

```

where "n - m" := (sub n m) : nat_scope.

```

Comparisons

```

Fixpoint eqb n m : bool :=
  match n, m with
  | 0, 0 => true
  | 0, S _ => false
  | S _, 0 => false
  | S n', S m' => eqb n' m'
  end.

```

```

Fixpoint leb n m : bool :=
  match n, m with
  | 0, _ => true
  | _, 0 => false
  | S n', S m' => leb n' m'
  end.

```

```

Definition ltb n m := leb (S n) m.

```

```

Infix "==" := eqb (at level 70) : nat_scope.
Infix "<=" := leb (at level 70) : nat_scope.
Infix "<?" := ltb (at level 70) : nat_scope.

```

```

Fixpoint compare n m : comparison :=
  match n, m with
  | 0, 0 => Eq
  | 0, S _ => Lt
  | S _, 0 => Gt
  | S n', S m' => compare n' m'
  end.

```

```

Infix "?=" := compare (at level 70) : nat_scope.

```

Minimum, maximum

```

Fixpoint max n m :=
  match n, m with
  | 0, _ => m
  | S n', 0 => n
  | S n', S m' => S (max n' m')
  end.

```

```

Fixpoint min n m :=
  match n, m with
  | 0, _ => 0
  | S n', 0 => 0
  | S n', S m' => S (min n' m')
  end.

```

Parity tests

```

Fixpoint even n : bool :=
  match n with
  | 0 => true
  | 1 => false
  | S (S n') => even n'
  end.

```

Definition odd n := negb (even n).

Power

```
Fixpoint pow n m :=
  match m with
  | 0 => 1
  | S m => n * (n^m)
  end
```

where "n ^ m" := (pow n m) : nat_scope.

Euclidean division

This division is linear and tail-recursive. In divmod, y is the predecessor of the actual divisor, and u is y minus the real remainder

```
Fixpoint divmod x y q u :=
  match x with
  | 0 => (q,u)
  | S x' => match u with
    | 0 => divmod x' y (S q) y
    | S u' => divmod x' y q u'
    end
  end.
```

```
Definition div x y :=
  match y with
  | 0 => y
  | S y' => fst (divmod x y' 0 y')
  end.
```

```
Definition modulo x y :=
  match y with
  | 0 => y
  | S y' => y' - snd (divmod x y' 0 y')
  end.
```

Infix "/" := div : nat_scope.

Infix "mod" := modulo (at level 40, no associativity) : nat_scope.

Greatest common divisor

We use Euclid algorithm, which is normally not structural, but Coq is now clever enough to accept this (behind modulo there is a subtraction,

which now preserves being a subterm)

```
Fixpoint gcd a b :=
  match a with
  | 0 => b
  | S a' => gcd (b mod (S a')) (S a')
  end.
```

Square

Definition square n := n * n.

Square root

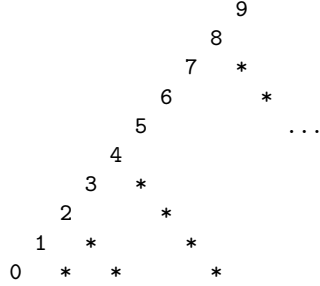
The following square root function is linear (and tail-recursive). With Peano representation, we can't do better. For faster algorithm, see Psqrt/Zsqrt/Nsqrt... We search the square root of $n = k + p^2 + (q - r)$ with $q = 2p$ and $0 \leq r \leq q$. We start with $p=q=r=0$, hence looking for the square root of $n = k$. Then we progressively decrease k and r . When $k = S k'$ and $r=0$, it means we can use $(S p)$ as new sqrt candidate, since $(S k') + p^2 + 2p = k' + (S p)^2$. When k reaches 0, we have found the biggest p^2 square contained in n , hence the square root of n is p .

```
Fixpoint sqrt_iter k p q r :=
  match k with
  | 0 => p
  | S k' => match r with
    | 0 => sqrt_iter k' (S p) (S (S q)) (S (S q))
    | S r' => sqrt_iter k' p q r'
  end
end.
```

Definition sqrt n := sqrt_iter n 0 0 0.

Log2

This base-2 logarithm is linear and tail-recursive. In `log2_iter`, we maintain the logarithm p of the counter q , while r is the distance between q and the next power of 2, more precisely $q + S r = 2^{(S p)}$ and $r < 2^p$. At each recursive call, q goes up while r goes down. When r is 0, we know that q has almost reached a power of 2, and we increase p at the next call, while resetting r to q . Graphically (numbers are q , stars are r) :



We stop when k , the global downward counter reaches 0. At that moment, q is the number we're considering (since $k+q$ is invariant), and p its logarithm.

```
Fixpoint log2_iter k p q r :=
  match k with
  | 0 => p
  | S k' => match r with
    | 0 => log2_iter k' (S p) (S q) q
    | S r' => log2_iter k' p (S q) r'
  end
end.
```

Definition `log2 n := log2_iter (pred n) 0 1 0.`

Iterator on natural numbers

```
Definition iter (n:nat) {A} (f:A->A) (x:A) : A :=
  nat_rect (fun _ => A) x (fun _ => f) n.
```

Bitwise operations We provide here some bitwise operations for unary numbers. Some might be really naive, they are just there for fulfilling the same interface as other for natural representations. As soon as binary representations such as `NArith` are available, it is clearly better to convert to/from them and use their ops.

```
Fixpoint div2 n :=
  match n with
  | 0 => 0
  | S 0 => 0
  | S (S n') => S (div2 n')
end.
```

```
Fixpoint testbit a n : bool :=
  match n with
  | 0 => odd a
  | S n => testbit (div2 a) n
end.
```

```

Definition shiftl a := nat_rect _ a (fun _ => double).
Definition shiftr a := nat_rect _ a (fun _ => div2).

```

```

Fixpoint bitwise (op:bool->bool->bool) n a b :=
  match n with
  | 0 => 0
  | S n' =>
    (if op (odd a) (odd b) then 1 else 0) +
    2*(bitwise op n' (div2 a) (div2 b))
  end.

```

```

Definition land a b := bitwise andb a a b.
Definition lor a b := bitwise orb (max a b) a b.
Definition ldiff a b := bitwise (fun b b' => andb b (negb b')) a a b.
Definition lxor a b := bitwise xorb (max a b) a b.

```


Chapter 10

Other Ideas to Explore

Computerising Mathematical Text[Kama15] explores various ways of capturing mathematical reasoning.

Chlipala[Chli15] gives a pragmatic approach to COQ.

Medina-Bulo et al.[Bulo04] gives a formal verification of Buchberger’s algorithm using ACL2 and Common Lisp.

Théry[Ther01] used COQ to check an implementation of Buchberger’s algorithm.

Pierce[Pier15] has a Software Foundations course in COQ with downloaded files in Pier15.tgz.

Spitters[Spit11] Type Classes for Mathematics in Coq. Also see <http://www.eelis.net/research/math-classes/>

Mahboubi[Mahb16] Mathematical Components. This book contains a proof of the Euclidean algorithm using COQ.

Aczel[Acze13] Homotopy Type Theory

Appendix A

The Global Environment

Let S be a set. Let $\circ$ be a binary operation. Let $+$ be an additive operation. Let $*$ be a multiplicative operation.

Axiom 1 (Magma) *A Magma is the set S with a closed binary operation $S \circ S \rightarrow S$ such that*

$$\forall a, b \in S \Rightarrow a \circ b \in S$$

.

Axiom 2 (Semigroup) *A Semigroup is a Magma with the operation $\circ$ that is associative such that*

$$\forall a, b, c \in S \Rightarrow (a \circ b) \circ c = a \circ (b \circ c)$$

Axiom 3 (Abelian Semigroup) *An Abelian Semigroup is a Semigroup with the operation $\circ$ that is commutative such that*

$$\forall a, b \in S \Rightarrow a \circ b = b \circ a$$

Axiom 4 (Monoid) *A Monoid is a Semigroup with an identity element $e \in S$ such that*

$$\forall a \in S \Rightarrow e \circ a = a \circ e = a$$

Axiom 5 (Group) *A Group is a Monoid with an inverse element $b \in S$ and an identity element $i \in S$ such that*

$$\forall a \in S \exists b \in S \Rightarrow a \circ b = b \circ a = i$$

Axiom 6 (Group Unique Identity) *A Group has a unique identity element $e \in S$ such that*

$$\exists e \wedge \forall a, b \in S \wedge a \neq e \wedge b \neq e \Rightarrow a \circ b \neq e$$

Axiom 7 (Group Unique Inverse) *A Group has a unique inverse element $i \in S$ such that*

$$\exists i \wedge \forall a, b \in S \wedge a \neq i \wedge b \neq i \Rightarrow a \circ b \neq i$$

Axiom 8 (Group Right Quotient) *A Group has a Right Quotient (right division) such that*

$$x \circ a = b \Rightarrow x \circ a \circ a^{-1} = b \circ a^{-1} \Rightarrow x = b \circ a^{-1}$$

Axiom 9 (Group Left Quotient) *A Group has a Left Quotient (left division) such that*

$$a \circ x = b \Rightarrow a^{-1} \circ a \circ x = a^{-1} \circ b \Rightarrow x = a^{-1} \circ b$$

Axiom 10 (Abelian Group) *An Abelian Group is a Group with the operation $\circ$ that is commutative such that*

$$\forall a, b \in S \Rightarrow a \circ b = b \circ a$$

Axiom 11 (Abelian Group Quotient) *An Abelian Group has a Quotient (division) such that*

$$a^{-1} \circ a \circ x = a \circ a^{-1} \circ x$$

Axiom 12 (Euclidean Domain) *Let R be an integral domain. Let f be a function from $R \setminus \{0\}$ to the NonNegativeInteger domain. If a and b are in R and b is nonzero, then there are q and r in R such that $a = bq + r$ and either $r = 0$ or $f(r) < f(b)$*

Bibliography

- [Aczel13] Peter et al. Aczel. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.
- Link:** <https://hott.github.io/book/nightly/hott-letter-1075-g3c53219.pdf>
- [Avig14] Jeremy Avigad. Lean proof of gcd, 2014.
- Link:** <http://github.com/leanprover/lean2/blob/master/library/data/nat/gcd.lean>
- [Avig16] Jeremy Avigad. Lean github repository, 2016.
- Link:** <http://github.com/leanprover>
- [Bold07] Sylvie Boldo and Jean-Christophe Filliatre. Formal verification of floating-point programs.
- Link:** <http://www-lipn.univ-paris13.fr/CerPAN/files/ARITH.pdf>
- [Bold07a] Sylvie Boldo and Jean-Christophe Filliatre. Formal verification of floating-point programs.
- Abstract:** This paper introduces a methodology to perform formal verification of floating-point C programs. It extends an existing tool for verification of C programs, Caduceus, with new annotations for specific floating-point arithmetic. The Caduceus first-order logic model for C programs is extended accordingly. Then verification conditions are obtained in the usual way and can be discharged interactively with the Coqa proof assistant, using an existing Coq formalization of floating-point arithmetic. This methodology is already implemented and has been successfully applied to several short floating-point programs, which are presented in this paper.
- Link:** <http://www.lri.fr/~filliatr/ftp/publis/caduceus-floats.pdf>

- [Bold11] Sylvie Boldo and Claude Marche. Formal verification of numerical programs: from c annotated programs to mechanical proofs. *Mathematics in Computer Science*, 5:377–393, 2011.

Abstract: Numerical programs may require a high level of guarantee. This can be achieved by applying formal methods, such as machine-checked proofs. But these tools handle mathematical theorems while we are interested in C code, in which numerical computations are performed using floating-point arithmetic, whereas proof tools typically handle exact real arithmetic. To achieve this high level of confidence on C programs, we use a chain of tools: Frama-C, its Jessie plugin, Why and provers among Coq, Gappa, Alt-Ergo, CVC3 and Z3. This approach requires the C program to be annotated; each function must be precisely specified, and we prove the correctness of the program by proving both that it meets its specifications and that no runtime error may occur. The purpose of this paper is to illustrate, on various examples, the features of this approach.

Link: <https://hal.archives-ouvertes.fr/hal-00777605/document>

- [Book102] Axiom Authors. *Volume 10.2: Axiom Algebra: Categories*. Axiom Project, 2016.

Link: <http://axiom-developer.org/axiom-website/bookvol10.2.pdf>

- [Book103] Axiom Authors. *Volume 10.3: Axiom Algebra: Domains*. Axiom Project, 2016.

Link: <http://axiom-developer.org/axiom-website/bookvol10.3.pdf>

- [Bres93] David Bressoud. Review of the problems of mathematics. *Math. Intell.*, 15(4):71–73, 1993.

- [Buch97] Bruno Buchberger. Mathematica: doing mathematics by computer? *Advances in the design of symbolic computation systems*, pages 2–20, 1997, 978-3-211-82844-1.

- [Bulo04] I. Medina-Bulo, F. Palomo-Lozano, J.A. Alonso-Jiménez, and J.L. Ruiz-Reina. Verified computer algebra in acl2. *ASIC 2004, LNAI 3249*, pages 171–184, 2004.

Abstract: In this paper, we present the formal verification of a Common Lisp implementation of Buchberger’s algorithm for computing Groebner bases of polynomial ideals. This work is carried out in the ACL2 system and shows how verified Computer Algebra can be achieved in an executable logic.

- [COQnat] COQ Proof Assistant. Library Coq.Init.Nat, 2017.
- Abstract:** Peano natural numbers, definitions of operations
- Link:** <https://coq.inria.fr/library/Coq.Init.Nat.html>
- [Chli15] Adam Chlipala. *Certified Programming with Dependent Types*. MIT Press, 2015, 9780262026659.
- Link:** <http://adam.chlipala.net/cpdt/cpdt.pdf>
- [Coqu16] Thierry Coquand, Gérard Huet, and Christine Paulin. The coq proof assistant, 2016.
- Link:** <https://coq.inria.fr>
- [Coqu16a] Thierry Coquand, Gérard Huet, and Christine Paulin. Coq proof assistant library coq.zarith.znumtheory, 2016.
- Link:** <https://coq.inria.fr/library/Coq.ZArith.Znumtheory.html>
- [Coqu86] Thierry Coquand and Gérard Huet. The calculus of constructions. Technical Report 530, INRIA Centre de Rocquencourt, 1986.
- Abstract:** The Calculus of Constructions is a higher-order formalism for constructive proofs in natural deduction style. Every proof is a λ -expression, typed with propositions of the underlying logic. By removing types we get a pure λ -expression, expressing its associated algorithm. Computing this λ -expression corresponds roughly to cut-elimination. It is our thesis that (as already advocated by Martin-Lof) the Curry-Howard correspondence between propositions and types is a powerful paradigm for Computer Science. In the case of Constructions, we obtain the notion of a very high-level functional programming language, with complex polymorphism well-suited for modules specification. The notion of type encompasses the usual notion of data type, but allows as well arbitrarily complex algorithmic specifications. We develop the basic theory of a Calculus of Constructions, and prove a strong normalization theorem showing that all computations terminate. Finally, we suggest various extensions to stronger calculi.
- Link:** <https://hal.inria.fr/inria-00076024/document>
- [Frad08] Maria Joao Frade. Calculus of inductive construction. software formal verification. *MFES*, 2008.
- Link:** <http://www4.di.uminho.pt/~jno/mfes/0809/SFV-CIC.pdf>

- [Hard13] David S. Hardin, Jedidiah R. McClurg, and Jennifer A. Davis. Creating formally verified components for layered assurance with an llvm to acl2 translator.

Abstract: This paper describes an effort to create a library of formally verified software component models from code that have been compiled using the Low-Level Virtual Machine (LLVM) intermediate form. The idea is to build a translator from LLVM to the applicative subset of Common Lisp accepted by the ACL2 theorem prover. They perform verification of the component model using ACL2's automated reasoning capabilities.

Link: http://www.jrmcclurg.com/papers/law_2013_paper.pdf

- [Hard14] David S. Hardin, Jennifer A. Davis, David A. Greve, and Jedidiah R. McClurg. Development of a translator from llvm to acl2.

Abstract: In our current work a library of formally verified software components is to be created, and assembled, using the Low-Level Virtual Machine (LLVM) intermediate form, into sub-systems whose top-level assurance relies on the assurance of the individual components. We have thus undertaken a project to build a translator from LLVM to the applicative subset of Common Lisp accepted by the ACL2 theorem prover. Our translator produces executable ACL2 formal models, allowing us to both prove theorems about the translated models as well as validate those models by testing. The resulting models can be translated and certified without user intervention, even for code with loops, thanks to the use of the `def::ung` macro which allows us to defer the question of termination. Initial measurements of concrete execution for translated LLVM functions indicate that performance is nearly 2.4 million LLVM instructions per second on a typical laptop computer. In this paper we overview the translation process and illustrate the translator's capabilities by way of a concrete example, including both a functional correctness theorem as well as a validation test for that example.

Link: <http://arxiv.org/pdf/1406.1566>

- [Jack95] Paul Bernard Jackson. *Enhancing the NUPRL Proof Development System and Applying it to Computational Abstract Algebra*. PhD thesis, Cornell University, 1 1995.

Abstract: This thesis describes substantial enhancements that were made to the software tools in the Nuprl system that are used to interactively guide the production of formal proofs. Over

20,000 lines of code were written for these tools. Also, a corpus of formal mathematics was created that consists of roughly 500 definitions and 1300 theorems. Much of this material is of a foundational nature and supports all current work in Nuprl. This thesis concentrates on describing the half of this corpus that is concerned with abstract algebra and that covers topics central to the mathematics of the computations carried out by computer algebra systems. The new proof tools include those that solve linear arithmetic problems, those that apply the properties of order relations, those that carry out inductive proof to support recursive definitions, and those that do sophisticated rewriting. The rewrite tools allow rewriting with relations of differing strengths and take care of selecting and applying appropriate congruence lemmas automatically. The rewrite relations can be order relations as well as equivalence relations. If they are order relations, appropriate monotonicity lemmas are selected. These proof tools were heavily used throughout the work on computational algebra. Many examples are given that illustrate their operation and demonstrate their effectiveness. The foundation for algebra introduced classes of monoids, groups, ring and modules, and included theories of order relations and permutations. Work on finite sets and multisets illustrates how a quotienting operation hides details of datatypes when reasoning about functional programs. Theories of summation operators were developed that drew indices from integer ranges, lists and multisets, and that summed over all the classes mentioned above. Elementary factorization theory was developed that characterized when cancellation monoids are factorial. An abstract data type for the operations of multivariate polynomial arithmetic was defined and the correctness of an implementation of these operations was verified. The implementation is similar to those found in current computer algebra systems. This work was all done in Nuprl's constructive type theory. The thesis discusses the appropriateness of this foundation, and the extent to which the work relied on it.

Keyword: axiomref

[Juds15] Thomas W. Judson. *Abstract Algebra: Theory and Applications*. Website, 2015.

Link: <http://abstract.ups.edu/aata/colophon-1.html>

[Kama15] Fairouz Kamareddine, Joe Wells, Christoph Zengler, and Henk Barendregt. *Computerising mathematical text*, 2015.

Abstract: Mathematical texts can be computerised in many ways that capture differing amounts of the mathematical mean-

ing. At one end, there is document imaging, which captures the arrangement of black marks on paper, while at the other end there are proof assistants (e.g. Mizar, Isabelle, Coq, etc.), which capture the full mathematical meaning and have proofs expressed in a formal foundation of mathematics. In between, there are computer typesetting systems (e.g. Latex and Presentation MathML) and semantically oriented systems (e.g. Content MathML, OpenMath, OMDoc, etc.). In this paper we advocate a style of computerisation of mathematical texts which is flexible enough to connect the different approaches to computerisation, which allows various degrees of formalisation, and which is compatible with different logical frameworks (e.g. set theory, category theory, type theory, etc.) and proof systems. The basic idea is to allow a man-machine collaboration which weaves human input with machine computation at every step in the way. We propose that the huge step from informal mathematics to fully formalised mathematics be divided into smaller steps, each of which is a fully developed method in which human input is minimal.

- [Lamp02] Leslie Lamport. *Specifying Systems*. Addison-Wesley, 2002, 0-321-14306-X.
- Link:** <http://research.microsoft.com/en-us/um/people/lamport/tla/book-02-08-08.pdf>
- [Lamp14] Leslie Lamport. How to write a 21st century proof, 2014.
- Abstract:** A method of writing proofs is described that makes it harder to prove things that are not true. The method, based on hierarchical structuring, is simple and practical. The author's twenty years of experience writing such proofs is discussed.
- Link:** <http://research.microsoft.com/en-us/um/people/lamport/pubs/paper.pdf>
- [Lamp14a] Leslie Lamport. Talk: How to write a 21st century proof, 2014.
- Comment:** 2nd Heidelberg Laureate Forum Lecture Tuesday Sep 23, 2014
- Link:** <http://hits.mediasite.com/mediasite/Play/29d825439b3c49f088d35555426fbdf81d>
- [Lamp16] Leslie Lamport. Tla+ proof system, 2016.
- Abstract:** Demonstration of Euclid Algorithm Proof in TLA+
- Link:** https://tla.msr-inria.inria.fr/tlaps/content/Documentation/Tutorial/The_example.html

- [Mahb16] Assia Mahboubi, Enrico Tassi, Yves Bertot, and Georges Gonthier. *Mathematical Components*. math-comp.github.io/mcb, 2016.

Abstract: *Mathematical Components* is the name of a library of formalized mathematic for the COQ system. It covers a variety of topics, from the theory of basic data structures (e.g. numbers, lists, finite sets) to advanced results in various flavors of algebra. This library constitutes the infrastructure for the machine-checked proofs of the Four Color Theorem and the Odd Order Theorem. The reason of existence of this books is to break down the barriers to entry. While there are several books around covering the usage of the COQ system and the theory it is based on, the Mathematical Components library is build in an unconventional way. As a consequence, this book provides a non-standard presentation of COQ, putting upfront the formalization choices and the proof style that are the pillars of the library. This book targets two classes of public. On one hand, newcomers, even the more mathematically inclined ones, find a soft introduction to the programming language of COQ, Gallina, and the Ssreflect proof language. On the other hand accustomed COQ users find a substantial account of the formalization style that made the Mathematical Components library possible. By no means does this book pretend to be a complete description of COQ or Ssreflect: both tools already come with a comprehensive user manual. In the course of the book, the reader is nevertheless invited to experiment with a large library of formalized concepts and she is given as soon as possible sufficient tools to prove non-trivial mathematical results by reusing parts of the library. By the end of the first part, the reader has learnt how to prove formally the infinitude of prime numbers, or the correctness of the Euclidean’s division algorithm, in a few lines of proof text.

Link: <https://math-comp.github.io/mcb/book.pdf>

- [Maso86] Ian A. Mason. *The Semantics of Destructive Lisp*. Center for the Study of Language and Information, 1986, 0-937073-06-7.

Abstract: Our basic premise is that the ability to construct and modify programs will not improve without a new and comprehensive look at the entire programming process. Past theoretical research, say, in the logic of programs, has tended to focus on methods for reasoning about individual programs; little has been done, it seems to us, to develop a sound understanding of the process of programming – the process by which programs evolve in concept and in practice. At present, we lack the means to describe the techniques of program construction and improve-

ment in ways that properly link verification, documentation and adaptability.

[OCAM14]

unknown. The ocaml website.

Link: <http://ocaml.org>

[Pfei12]

Greg Pfeil. Common lisp type hierarchy, 2012.

Link: <http://sellout.github.io/2012/03/03/common-lisp-type-hierarchy>

[Pier15]

Benjamin C. Pierce, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Catalin Hritcu, Vilhelm Sjöberg, and Brent Yorgey. Software foundations, 2015.

Abstract: This electronic book is a course on Software Foundations, the mathematical underpinnings of reliable software. Topics include basic concepts of logic, computer-assisted theorem proving, the Coq proof assistant, functional programming, operational semantics, Hoare logic, and static type systems. The exposition is intended for a broad range of readers, from advanced undergraduates to PhD students and researchers. No specific background in logic or programming languages is assumed, though a degree of mathematical maturity will be helpful. The principal novelty of the course is that it is one hundred per cent formalized and machine-checked: the entire text is literally a script for Coq. It is intended to be read alongside an interactive session with Coq. All the details in the text are fully formalized in Coq, and the exercises are designed to be worked using Coq. The files are organized into a sequence of core chapters, covering about one semester’s worth of material and organized into a coherent linear narrative, plus a number of appendices covering additional topics. All the core chapters are suitable for both upper-level undergraduate and graduate students.

[Sho08]

Victor Shoup. A computational introduction to number theory.

Link: <http://shoup.net/ntb/ntb-v2.pdf>

[Spit11]

Bas Spitters and Eelis van der Weegen. Type classes for mathematics in type theory. *Math. Struct. Comput. Sci.*, 21(4):795–825, 2011.

Abstract: The introduction of first-class type classes in the Coq system calls for a re-examination of the basic interfaces used for mathematical formalisation in type theory. We present a new set of type classes for mathematics and take full advantage of their unique features to make practical a particularly flexible approach that was formerly thought to be infeasible. Thus, we

address traditional proof engineering challenges as well as new ones resulting from our ambition to build upon this development a library of constructive analysis in which any abstraction penalties inhibiting efficient computation are reduced to a minimum. The basis of our development consists of type classes representing a standard algebraic hierarchy, as well as portions of category theory and universal algebra. On this foundation, we build a set of mathematically sound abstract interfaces for different kinds of numbers, succinctly expressed using categorical language and universal algebra constructions. Strategic use of type classes lets us support these high-level theory-friendly definitions, while still enabling efficient implementations unhindered by gratuitous indirection, conversion or projection. Algebra thrives on the interplay between syntax and semantics. The Prolog-like abilities of type class instance resolution allow us to conveniently define a quote function, thus facilitating the use of reflective techniques.

Link: <https://arxiv.org/pdf/1102.1323.pdf>

- [Stac17] cstheory.stackexchange.com. Why does coq have prop?, 2017.
- Link:** <http://cstheory.stackexchange.com/questions/21836/why-does-coq-have-prop/21878#21878>
- [Ther01] Laurent Théry. A machine-checked implementation of buchberger’s algorithm. *Journal of Automated Reasoning*, 26:107–137, 2001.
- Abstract:** We present an implementation of Buchberger’s algorithm that has been proved correct within the proof assistant Coq. The implementation contains the basic algorithm plus two standard optimizations.
- [Thur94] William P. Thurston. On proof and progress in mathematics. *Bulletin AMS*, 30(2), April 1994.
- Link:** <http://www.ams.org/journals/bull/1994-30-02/S0273-0979-1994-00502-6/S0273-0979-1994-00502-6.pdf>
- [Wiki14a] ProofWiki. Euclidean algorithm.
- Link:** http://proofwiki.org/wiki/Euclidean_Algorithm
- [Wiki14b] ProofWiki. Division theorem.
- Link:** http://proofwiki.org/wiki/Division_Theorem
- [Wiki17] Wikipedia. Calculus of constructions, 2017.
- Link:** https://en.wikipedia.org/wiki/Calculus_of_constructions

- [WikiED] Wikipedia. Euclidean domain, 2017.
Link: https://en.wikipedia.org/wiki/Euclidean_domain
- [Zdan14] Steve Zdancewic and Milo M.K. Martin. Vellvm: Verifying the llvm.
Link: <http://www.cis.upenn.edu/~stevez/vellvm>

Index

common divisor, [12](#)

greatest common divisor, [12](#)

relatively prime, [12](#)

The Euclidean Algorithm, [12](#)